DAY 1 · THURSDAY 22 OCTOBER

Retrieval-Augmented Generation

Naïve RAG, better retrieval, Self-RAG and Corrective RAG · optional: fine-tuning



Pavlos Protopapas
Harvard University

Guillermo Cabrera-Vives
Universidad de Concepción

Outline

- Introduction - RAG
- Naïve RAG
- Pre-Retrieval Optimization
- Retrieval Optimization – Hybrid Search
- Post-Retrieval Optimization – Re-ranking
- Self-RAG
- Corrective-RAG
- Fine-tuning (PEFT, LoRA, QLoRA)

Introduction - RAG

LLMs ISSUES?

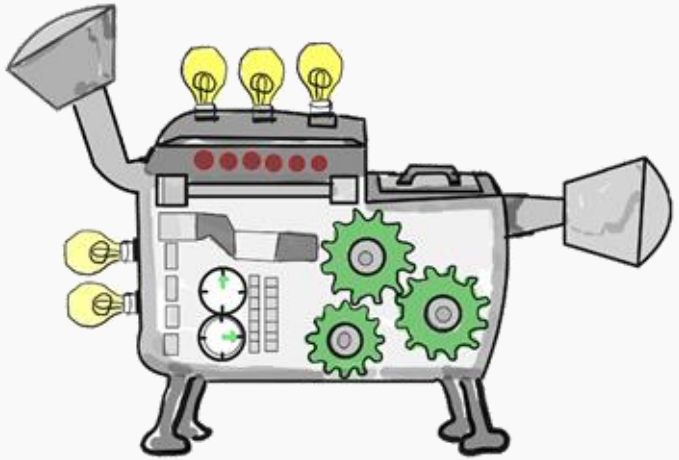
- Training data is not frequently updated
- Lack domain knowledge in specialized use cases
- Hallucinations
- Don’t cite the sources used to generate a response

Introduction - RAG

If we have many documents, how can we process/query them using an LLM?

1. Pass all the text directly into an LLM





LLM



If the context is too big, the LLM gives out garbage.

Garbage

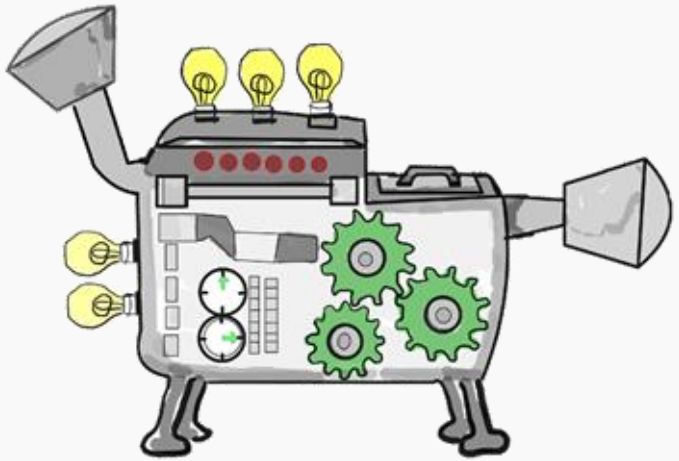
Introduction - RAG

If we have many documents, how can we process/query them using an LLM?

2. We can finetune our LLM using the data.

We will talk about fine-tuning at the end of this lecture.





LLM

Takes too long!

Introduction - RAG

If we have many documents, how can we process/query them using an LLM?

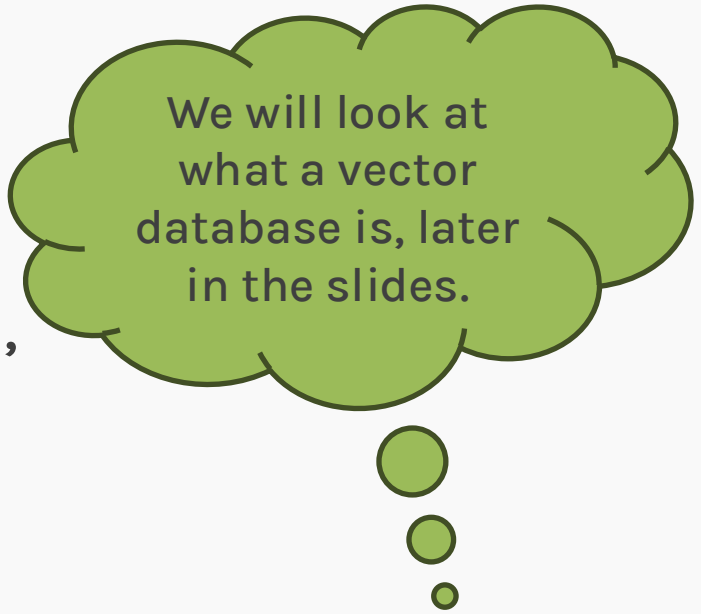
Let’s take a deeper look at what RAG is and how it can help us.

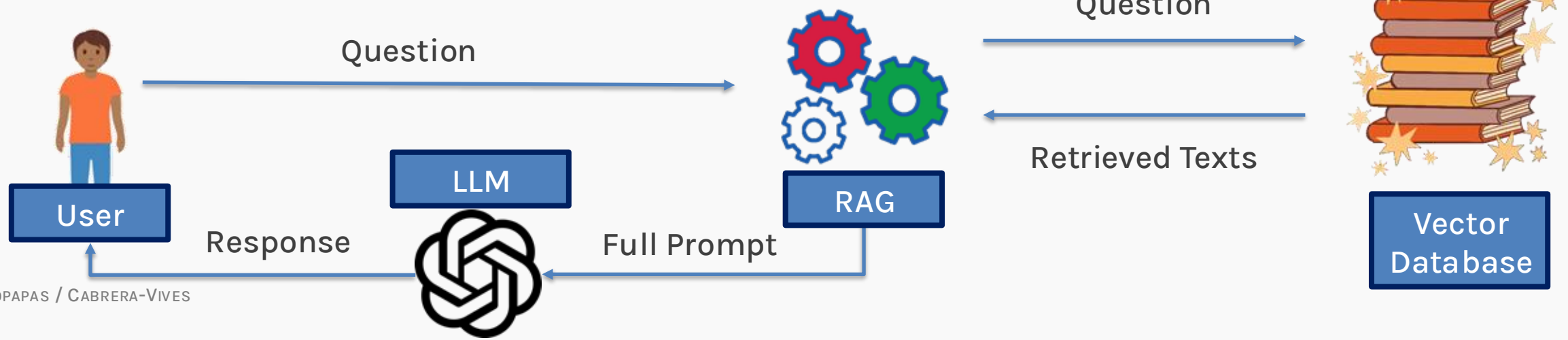


Introduction - RAG

What is RAG?

- RAG stands for Retrieval-Augmented-Generation.
- It is technique that improves the performance of a LLM, especially for tasks that require accurate and detailed information.
- A simple breakdown of how it works:





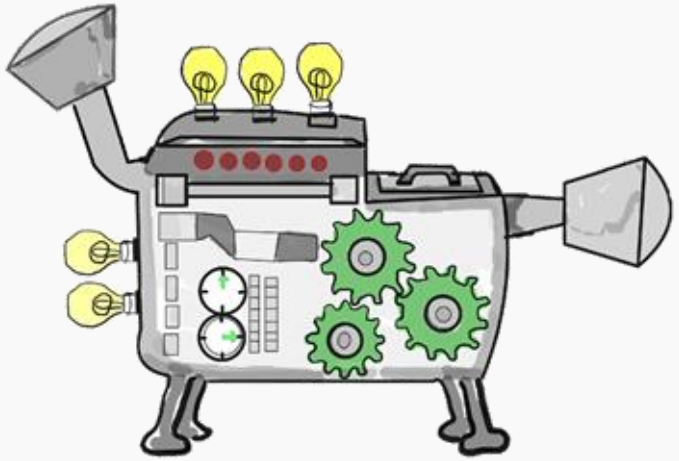
Introduction - RAG

What is RAG?

Query: What are different techniques of cheese making?

Key Concepts and Techniques: Central to cheese making are concepts like pasteurization to kill harmful bacteria, the use of rennet to coagulate milk, and the role of aging to develop flavors. Techniques like blending different milk types (cow, goat, sheep) and adding herbs or spices also contribute to unique characteristics.

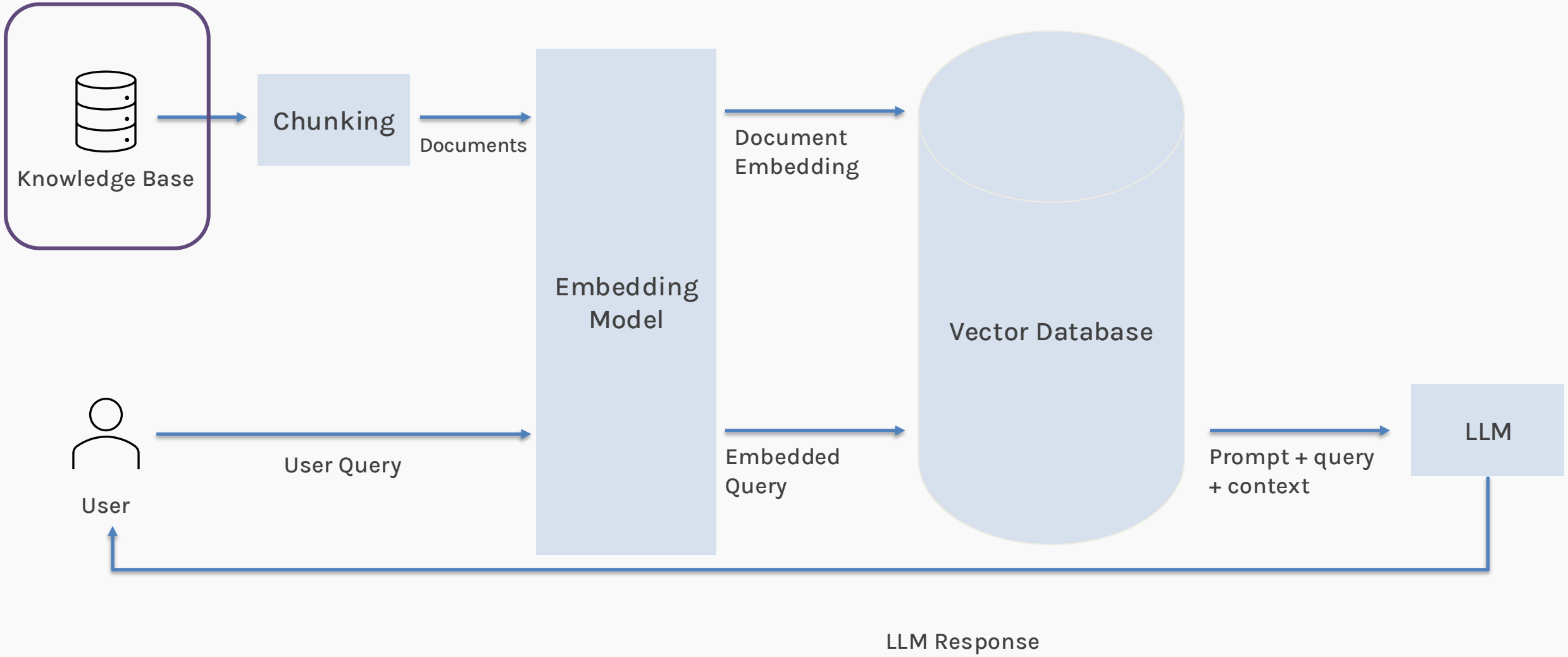
Context



LLM

The different techniques of cheese making are ....

Naïve RAG


Knowledge Base
Chunking
Documents
Embedding Model
Document Embedding
Vector Database
User
User Query
Embedded Query
Prompt + query + context
LLM
LLM Response

Naïve RAG – Knowledge Base

This could be any data source that contains useful information.



Websites



PDF Files



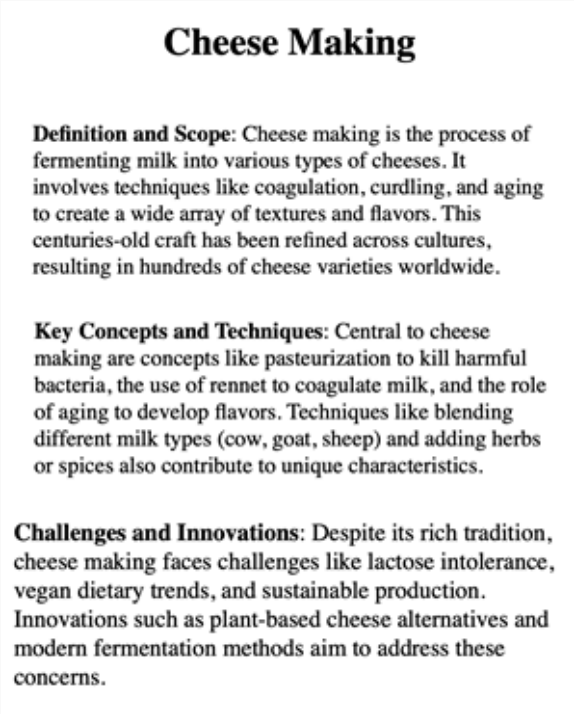
Word Files



Naïve RAG - Chunking

Chunking refers to the process of breaking down or segmenting information into smaller, more manageable pieces.

Chunking



Cheese Making

Definition and Scope: Cheese making is the process of fermenting milk into various types of cheeses. It involves techniques like coagulation, curdling, and aging to create a wide array of textures and flavors. This centuries-old craft has been refined across cultures, resulting in hundreds of cheese varieties worldwide.

Key Concepts and Techniques: Central to cheese making are concepts like pasteurization to kill harmful bacteria, the use of rennet to coagulate milk, and the role of aging to develop flavors. Techniques like blending different milk types (cow, goat, sheep) and adding herbs or spices also contribute to unique characteristics.

Challenges and Innovations: Despite its rich tradition, cheese making faces challenges like lactose intolerance, vegan dietary trends, and sustainable production. Innovations such as plant-based cheese alternatives and modern fermentation methods aim to address these concerns.

Naïve RAG

The embedding model converts text (documents and queries) into vectors/embeddings that capture semantic meaning.

Embedding Model

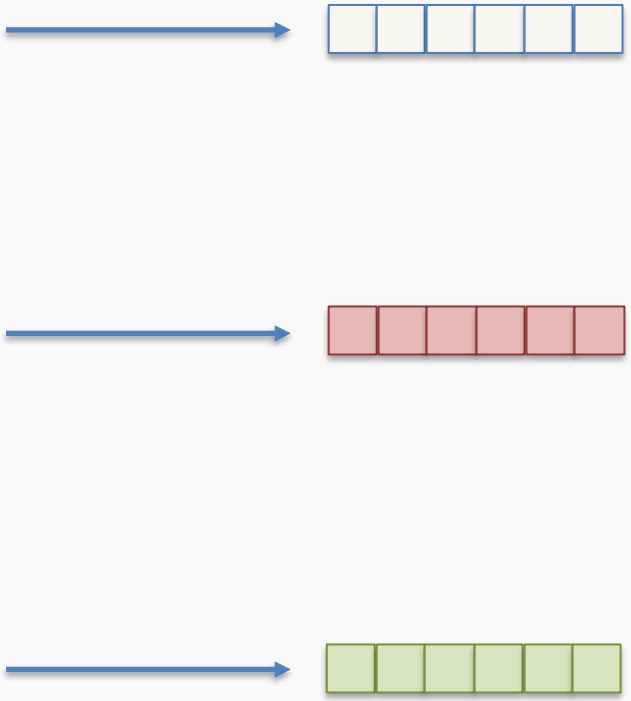
Cheese Making

Definition and Scope: Cheese making is the process of fermenting milk into various types of cheeses. It involves techniques like coagulation, curdling, and aging to create a wide array of textures and flavors. This centuries-old craft has been refined across cultures, resulting in hundreds of cheese varieties worldwide.

Key Concepts and Techniques: Central to cheese making are concepts like pasteurization to kill harmful bacteria, the use of rennet to coagulate milk, and the role of aging to develop flavors. Techniques like blending different milk types (cow, goat, sheep) and adding herbs or spices also contribute to unique characteristics.

Challenges and Innovations: Despite its rich tradition, cheese making faces challenges like lactose intolerance, vegan dietary trends, and sustainable production. Innovations such as plant-based cheese alternatives and modern fermentation methods aim to address these concerns.

Embeddings



Naïve RAG

The embedding model converts text (docu
into vectors/embeddings that capture sem

We use an architecture similar to BERT and aggregate the word embeddings

Embedding Model

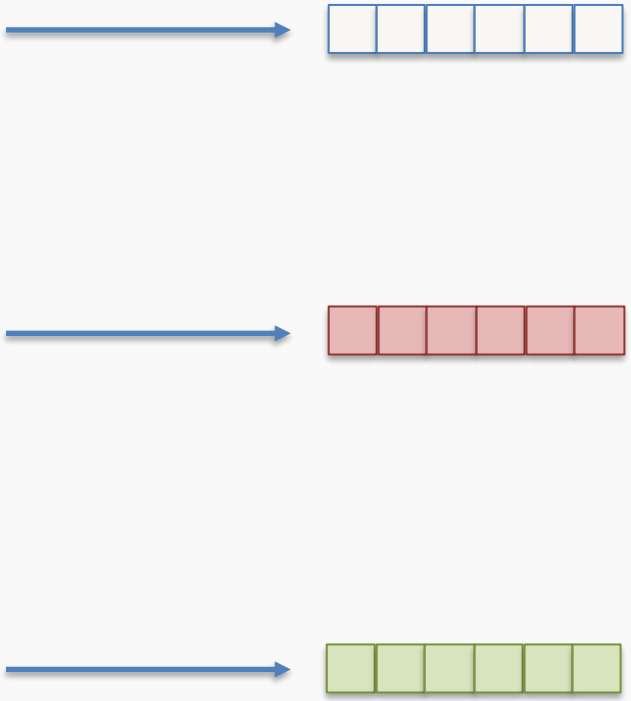
Cheese Making

Definition and Scope: Cheese making is the process of fermenting milk into various types of cheeses. It involves techniques like coagulation, curdling, and aging to create a wide array of textures and flavors. This centuries-old craft has been refined across cultures, resulting in hundreds of cheese varieties worldwide.

Key Concepts and Techniques: Central to cheese making are concepts like pasteurization to kill harmful bacteria, the use of rennet to coagulate milk, and the role of aging to develop flavors. Techniques like blending different milk types (cow, goat, sheep) and adding herbs or spices also contribute to unique characteristics.

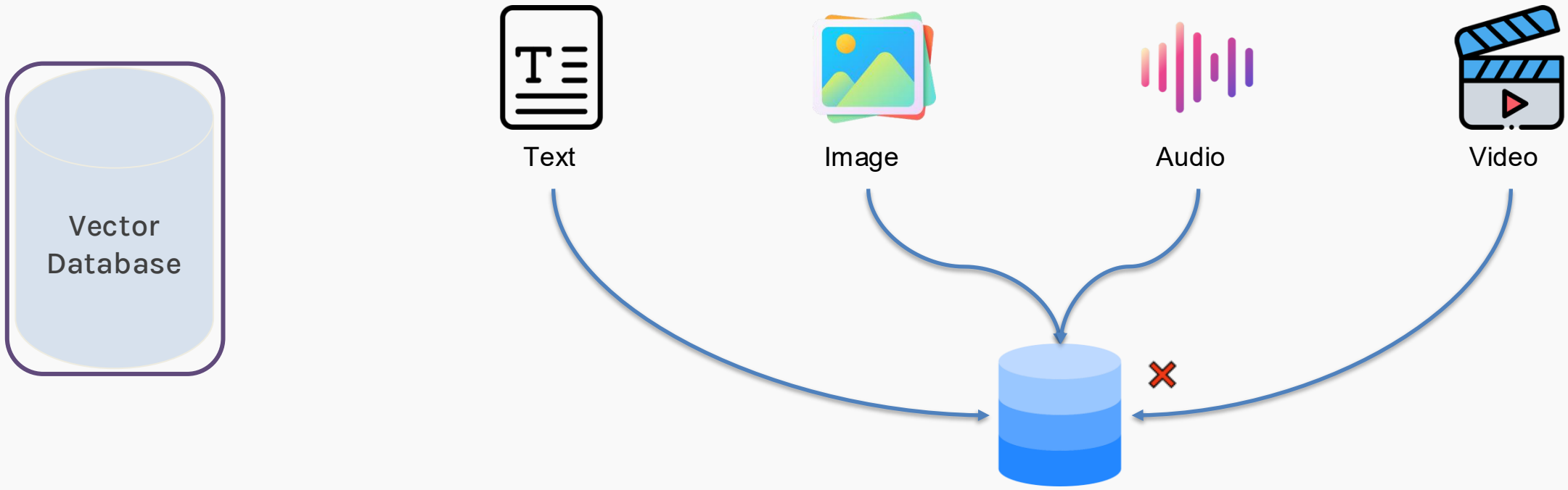
Challenges and Innovations: Despite its rich tradition, cheese making faces challenges like lactose intolerance, vegan dietary trends, and sustainable production. Innovations such as plant-based cheese alternatives and modern fermentation methods aim to address these concerns.

Embeddings



Naïve RAG – Vector Database

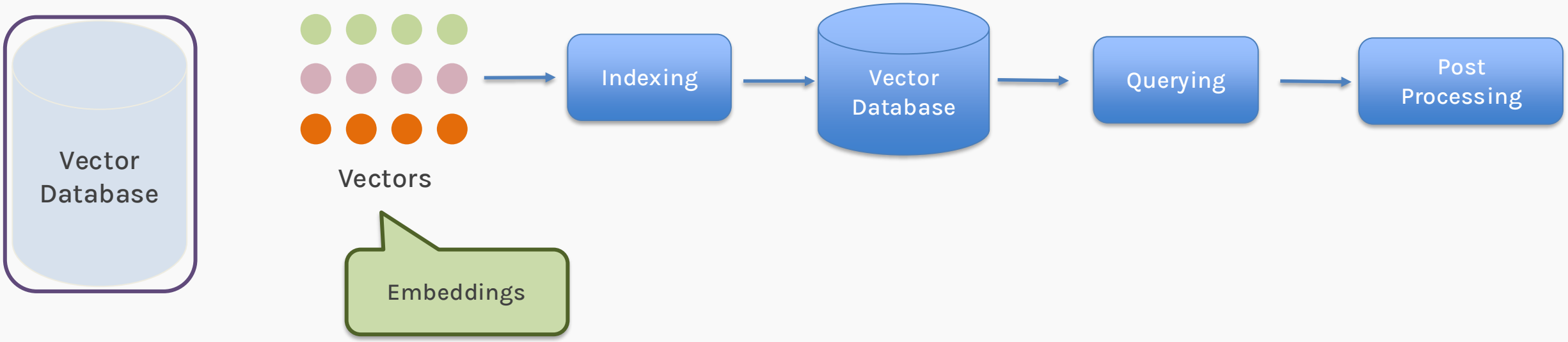
More than **80%** of the available data on the internet is unstructured.



We **can't** use the traditional **relational database** to store and later retrieve any particular/similar images as these are not structured and lack any metadata. Same goes with vectors!

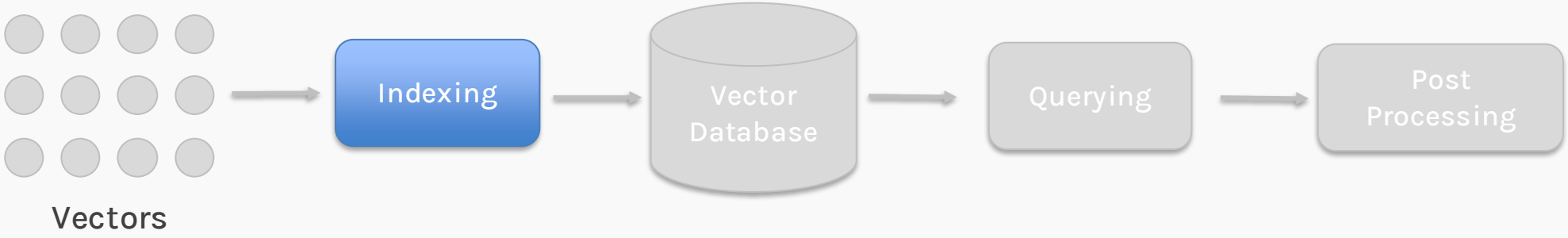
Naïve RAG – Vector Database

A vector database indexes and stores vector embeddings for fast retrieval and similarity search.



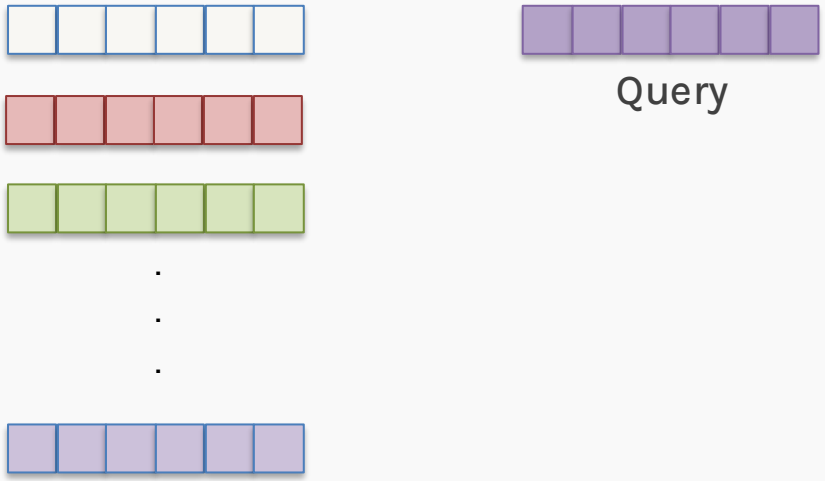
Vectors capture the essential features of the original data in a high-dimensional space.

Naïve RAG – Vector Database

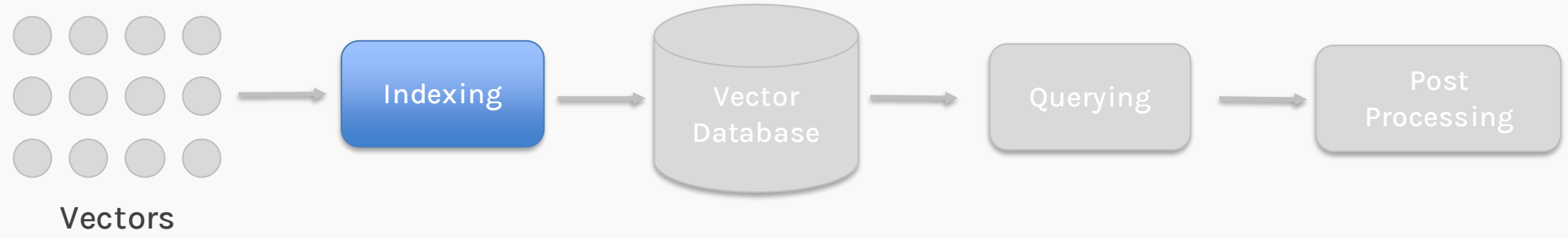


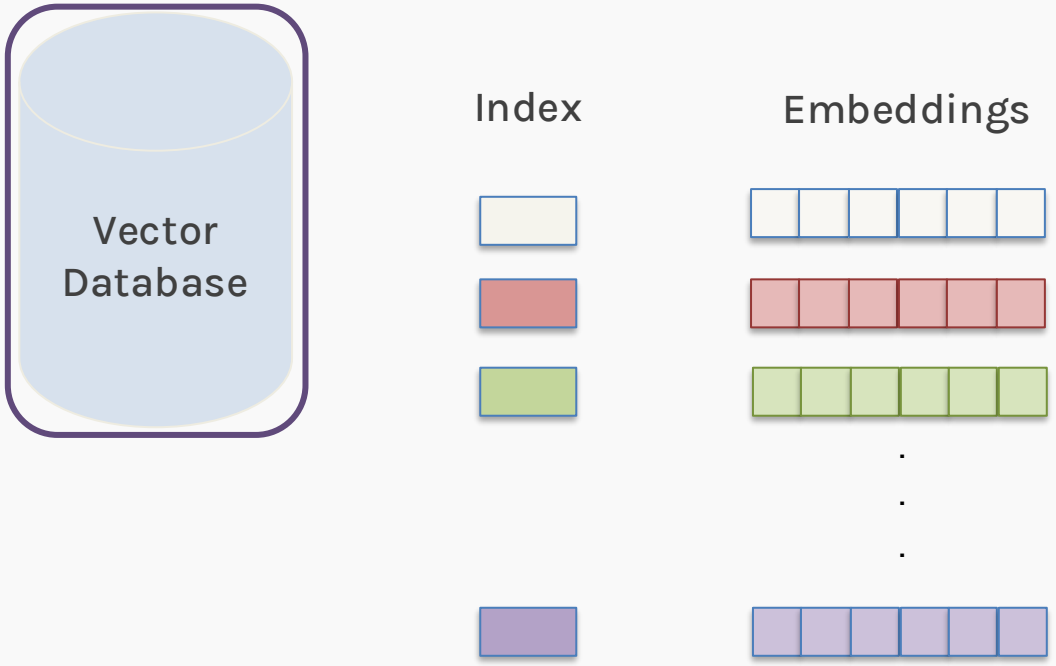


In reality, we could have millions of vectors to deal with.



Naïve RAG – Vector Database

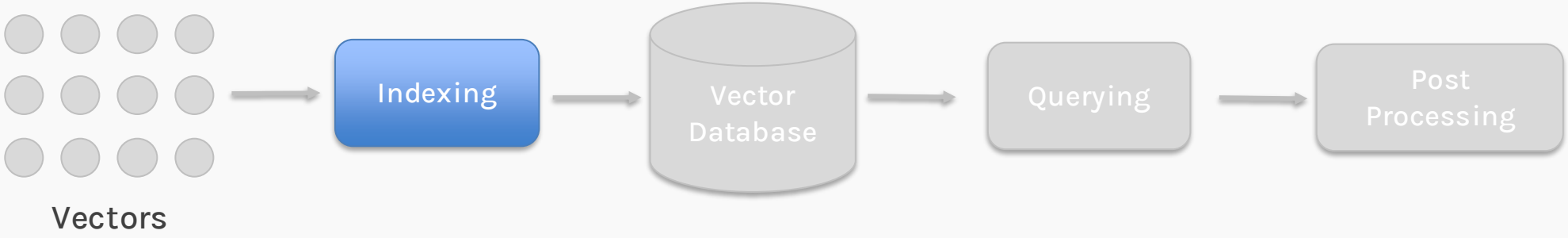




Indexing is the process of mapping the vectors to a data structure that will enable us to search faster.

When we query for a particular vector, the index helps us quickly zoom into a small subset of vectors that are likely to be similar.

Naïve RAG – Vector Database



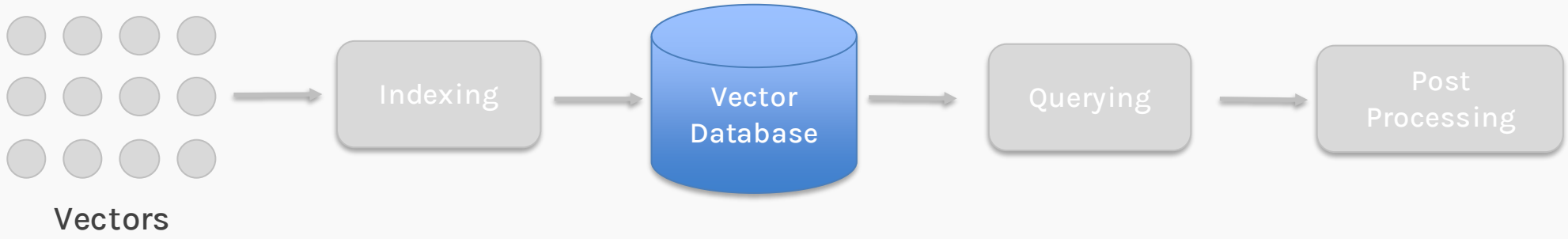


We usually use one of the three algorithms to do indexing:

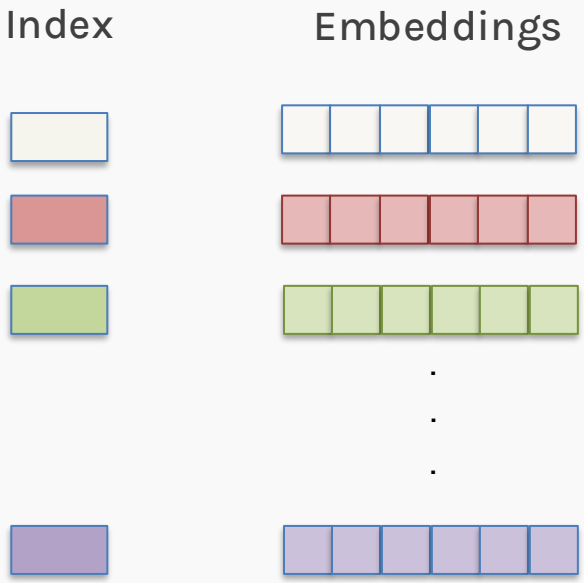
1. Hashing (Locality Sensitive Hashing - LSH)
2. Quantization (Product Quantization - PQ)
3. Graph Based (Hierarchical Navigable Small World - HNSW)

Nowadays, most relational databases like PostgreSQL have introduced vector as a column datatype, providing vector database benefits within traditional databases.

Naïve RAG – Vector Database

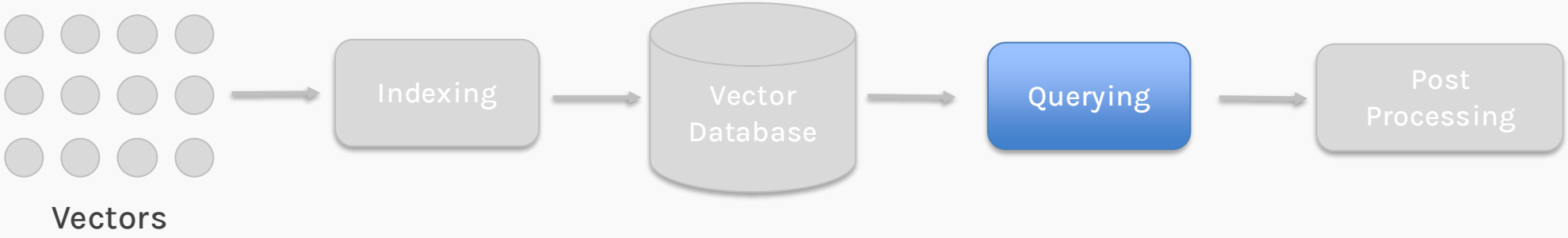






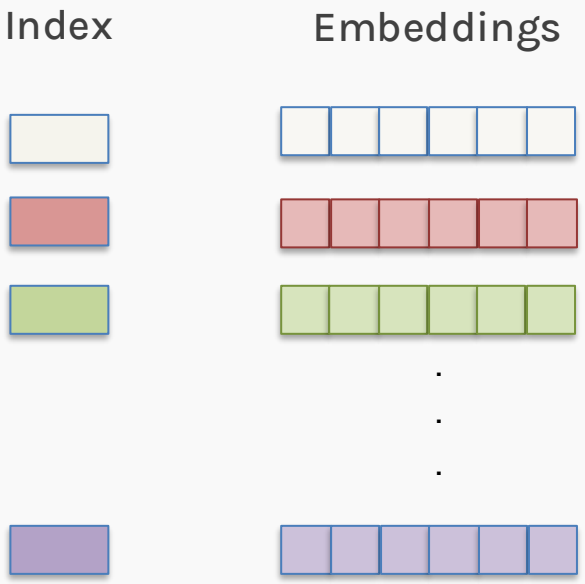
- A vector database stores indexed vectors, enabling fast similarity searches.
- These are optimized to handle such vectors and performing various search operations.

Naïve RAG – Vector Database

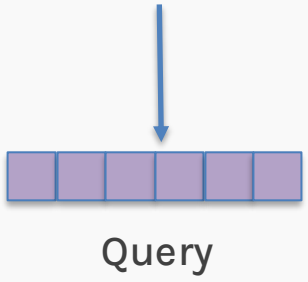




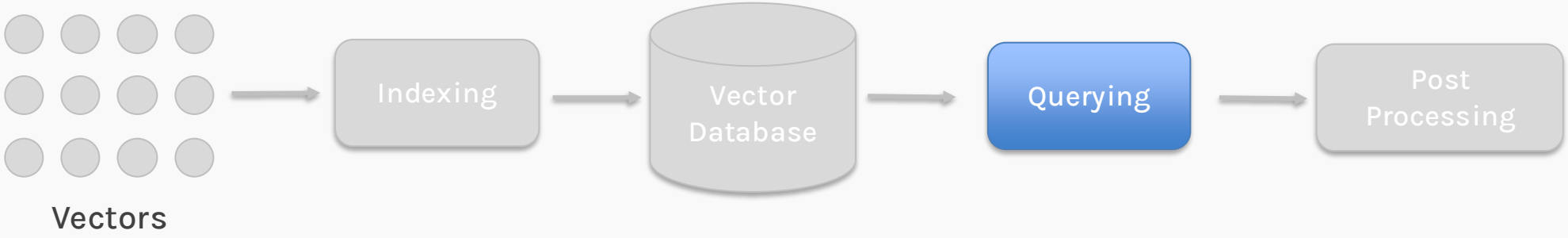
When querying, the vector database compares the indexed vectors to the query vector to determine the nearest vector neighbor.



Query: What are different techniques of cheese making?

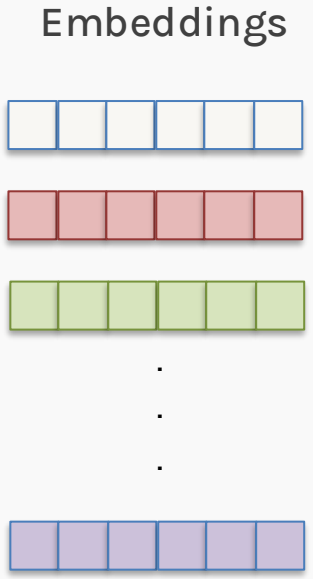


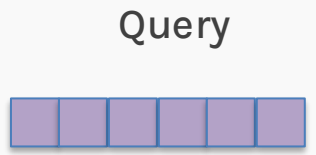
Naïve RAG – Vector Database



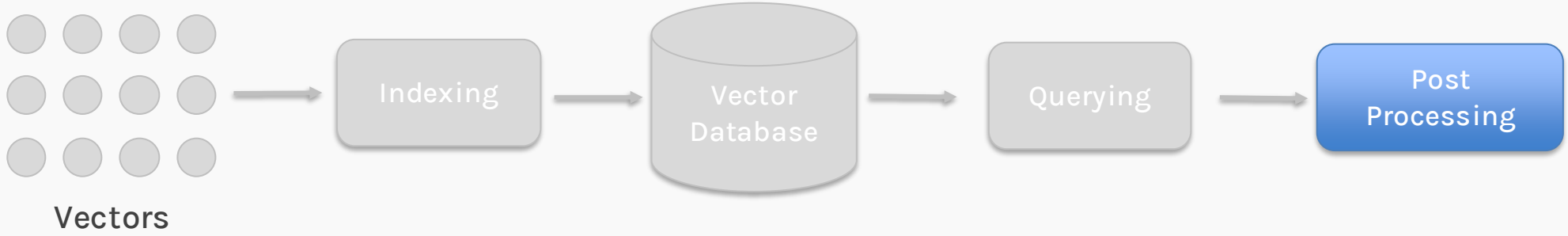


This leads to a ranking of embeddings based on their similarity with the query embedding.





Naïve RAG – Vector Database





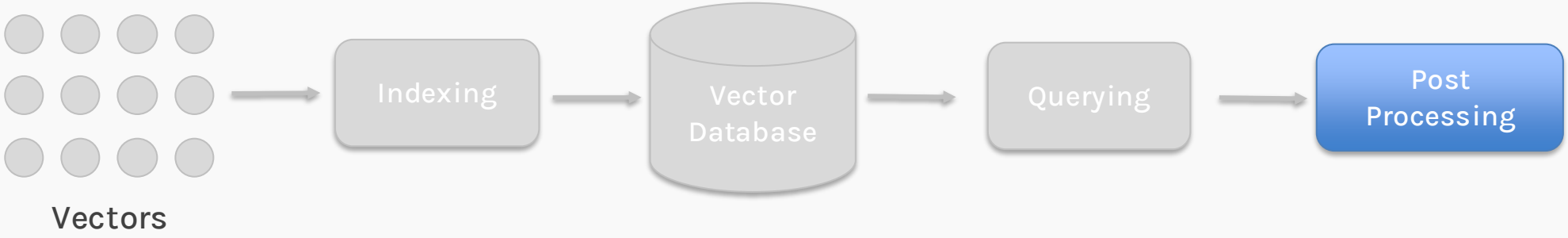
In advanced RAG, an additional pre-processing step re-ranks the retrieved results using a different similarity measure, improving the quality of inputs for the next stage.

We could also do some metadata filtering here.

Note: *This is an* ***optional*** *step.*

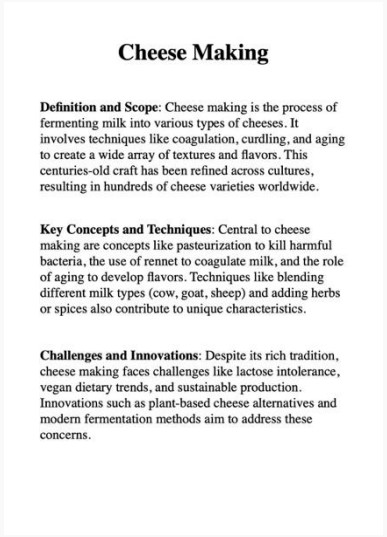
Metadata is data that provides information about other data (in our case, the vectors).

Naïve RAG – Vector Database





Metadata filtering helps refine and narrow down the search results based on metadata.



Cheese Making

Definition and Scope: Cheese making is the process of fermenting milk into various types of cheeses. It involves techniques like coagulation, curdling, and aging to create a wide array of textures and flavors. This centuries-old craft has been refined across cultures, resulting in hundreds of cheese varieties worldwide.

Key Concepts and Techniques: Central to cheese making are concepts like pasteurization to kill harmful bacteria, the use of rennet to coagulate milk, and the role of aging to develop flavors. Techniques like blending different milk types (cow, goat, sheep) and adding herbs or spices also contribute to unique characteristics.

Challenges and Innovations: Despite its rich tradition, cheese making faces challenges like lactose intolerance, vegan dietary trends, and sustainable production. Innovations such as plant-based cheese alternatives and modern fermentation methods aim to address these concerns.

For this data, the metadata may be:

1. Author
2. Publication date
3. Category

Naïve RAG – Prompt + query + context

Prompt

System prompts acts as an instruction given to the model to guide its behavior and responses

Query

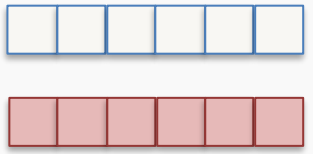
The query is the user's initial input.



Query

Context

Context refers to the information retrieved from the vector database that is relevant to the query,



Context

Naïve RAG – Prompt + query + context

Let's see an example of how this would look like:

System Prompt

"You are a Cheese Expert. When answering user questions explain in detail the key techniques and concepts involved in cheese making"

User Query

"What are the main steps and methods used in the cheese making process?"

Context (Relevant Extracts)

Central to cheese making are concepts like pasteurization, the use of rennet to coagulate milk, and the role of aging in developing flavors. Techniques such as blending different milk types (cow, goat, sheep) and adding herbs/spices contribute to unique cheese characteristics.

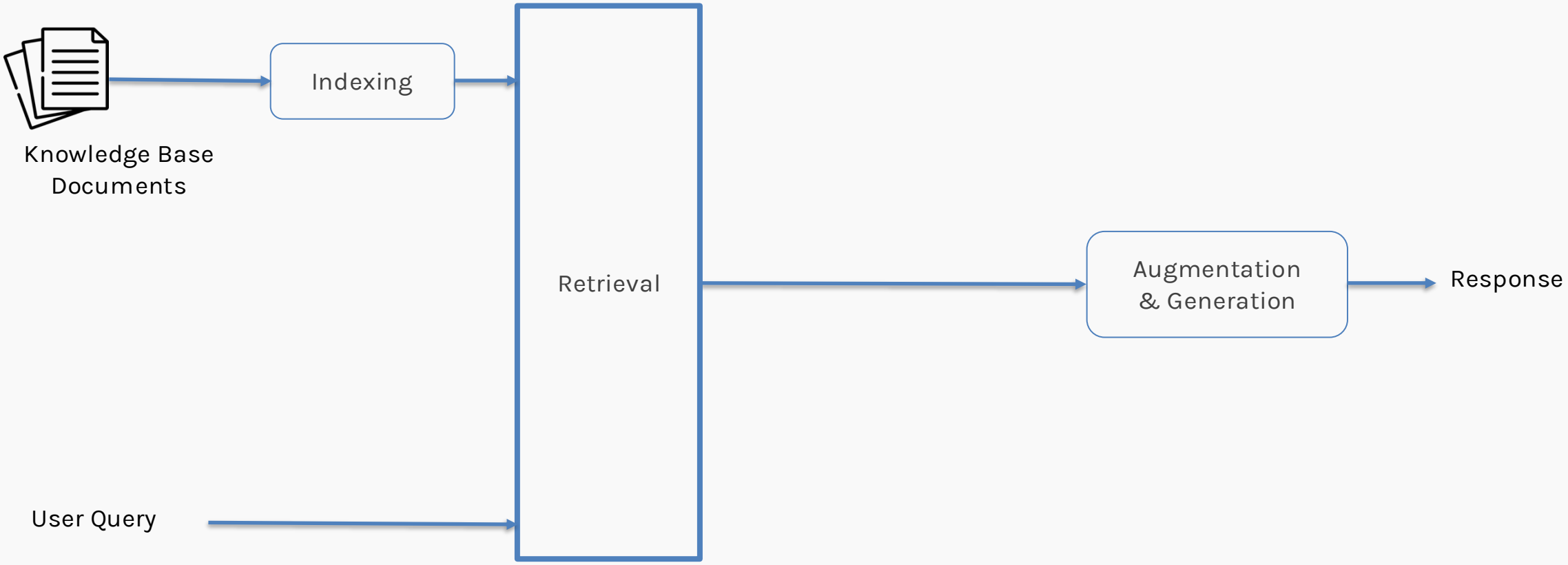
Final Prompt

Outline

- Introduction - RAG
- Naïve RAG
- **Pre-Retrieval Optimization**
- Retrieval Optimization – Hybrid Search
- Post-Retrieval Optimization – Re-ranking
- Self-RAG
- Corrective-RAG
- Fine-tuning (PEFT, LoRA, QLoRA)

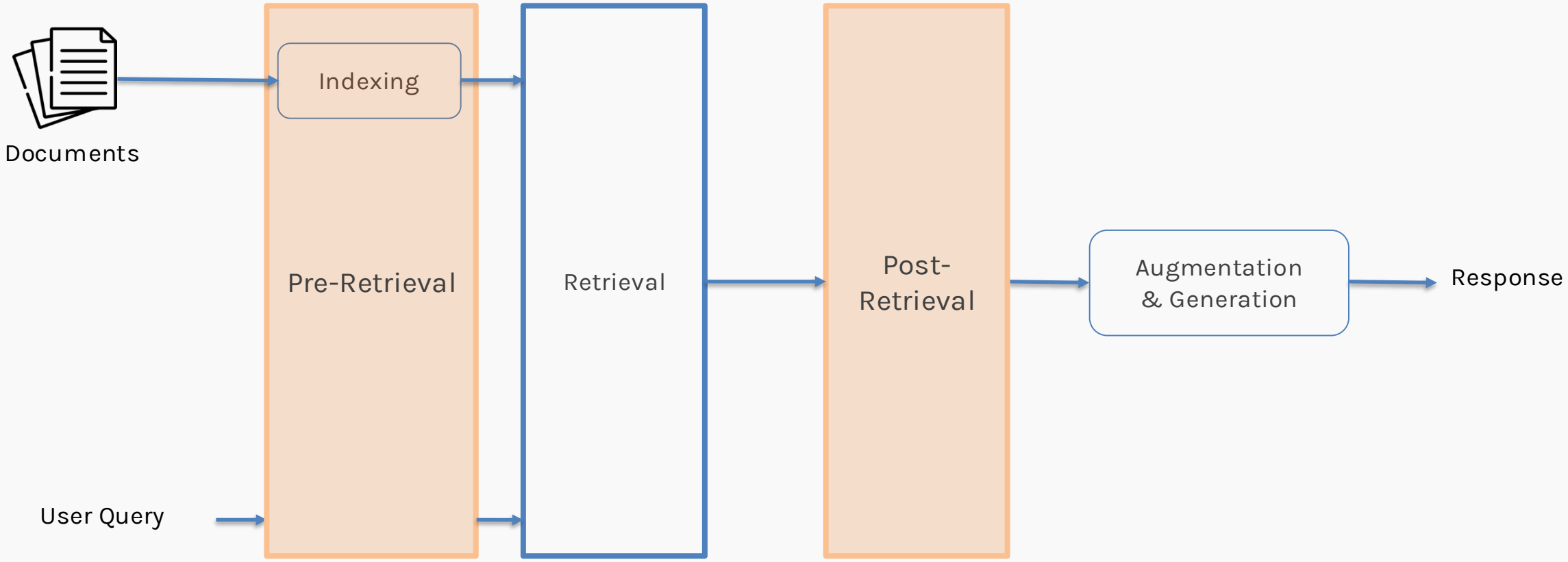
Naïve RAG

Now, let's look at the big picture.



Naïve RAG

Now, let's look at the big picture.



Pre-Retrieval Optimization

The pre-retrieval stage can be optimized in many ways.

We will be looking at two main ways of doing so:

1. Indexing
2. Query Manipulation

Pre-Retrieval Optimization - Indexing

Indexing improves the chunking process

By default, we do character splitting for the chunks. For example, if we have a document that says:

> Machine learning is a subset of artificial intelligence that focuses on building systems that learn from data. It is used in various applications such as recommendation engines, autonomous vehicles, and predictive analytics.

Pre-Retrieval Optimization - Indexing

Indexing improves the chunking process

> Machine learning is a subset of artificial intelligence that focuses on building systems that learn from data. It is used in various applications such as recommendation engines, autonomous vehicles, and predictive analytics.

By using character splitting of chunk size 50, the chunks would be:

a. "Machine learning is a subset of artificial intelli”
b. "gence that focuses on building systems that lear”
c. "n from data. It is used in various applications ”
d. "such as recommendation engines, autonomous vehic"
e. "les, and predictive analytics."



Pre-Retrieval Optimization - Indexing

Semantic Chunking – Steps

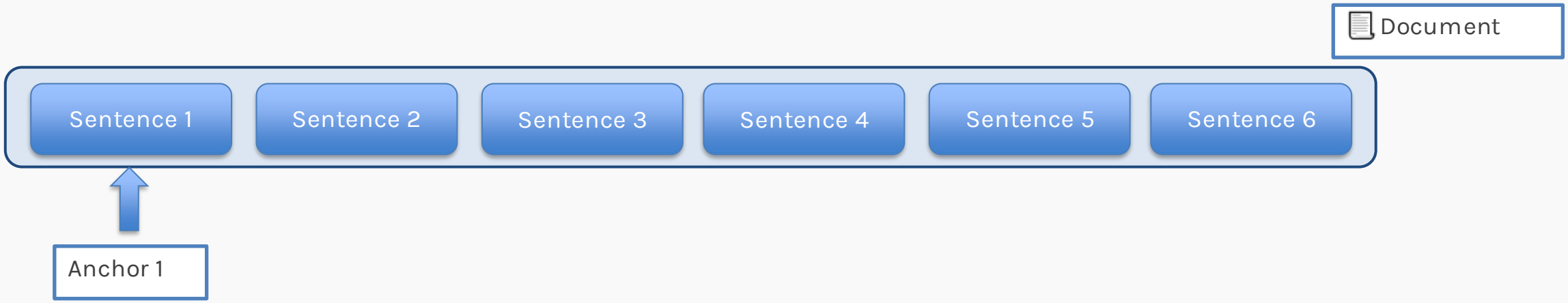
1. Splitting: We split the document to sentences using separators (.,?,!).

2. Grouping: Select anchor sentences and choose how many sentences to consider at either side of the anchor (window size).

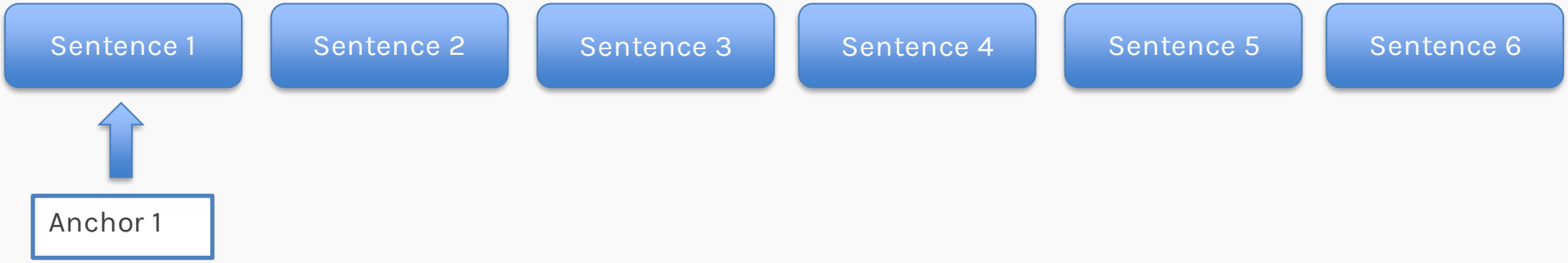
3. Similarity Check: Calculate the distance between the group of sentences (e.g.: cosine similarity on the embeddings).

4. Chunking: Chunk together the similar sentences.

Pre-Retrieval Optimization - Indexing

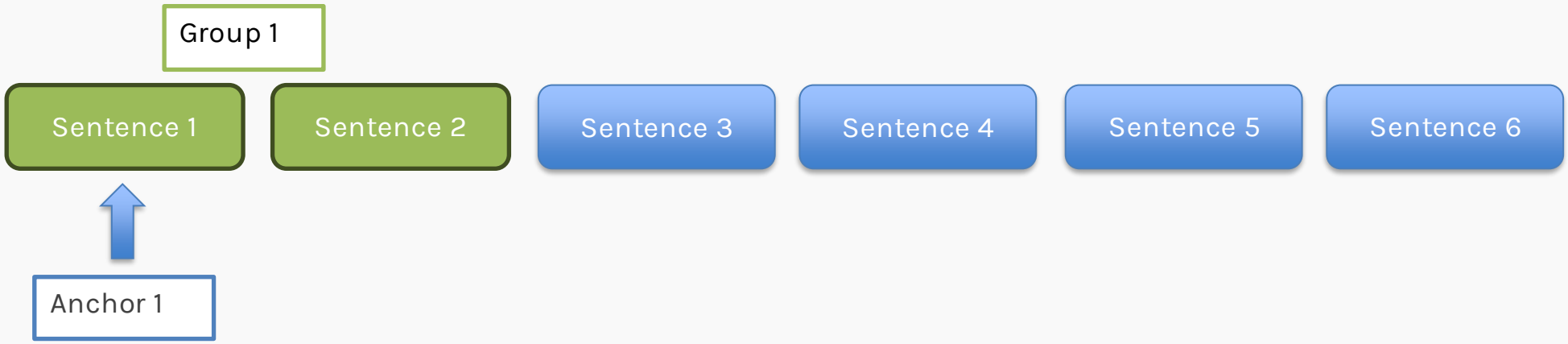


Pre-Retrieval Optimization - Indexing



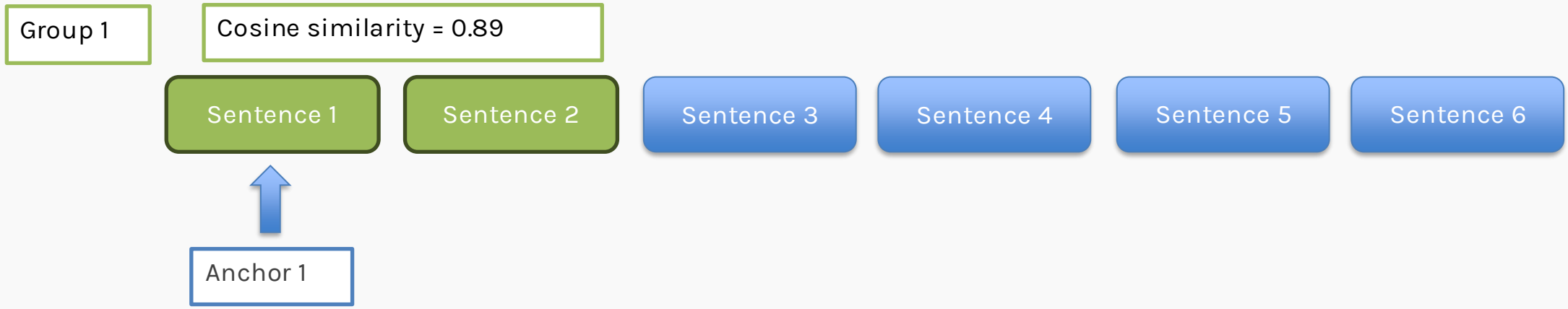
1. Let's suppose we choose the number of sentences at either side to be **1**, this is the window size.

Pre-Retrieval Optimization - Indexing



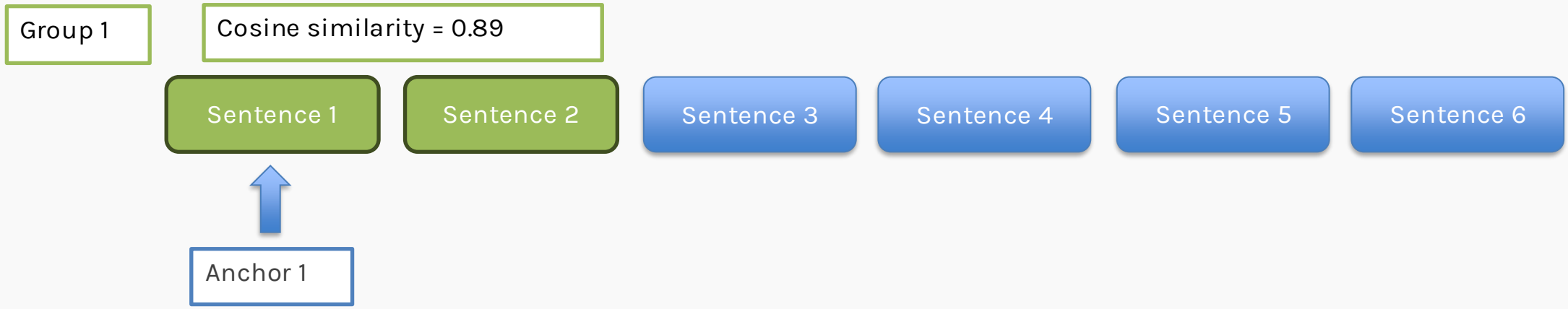
1. Let's suppose we choose the number of sentences at either side to be **1**, this is the window size.
2. We calculate the cosine similarity of the sentences in the group.

Pre-Retrieval Optimization - Indexing



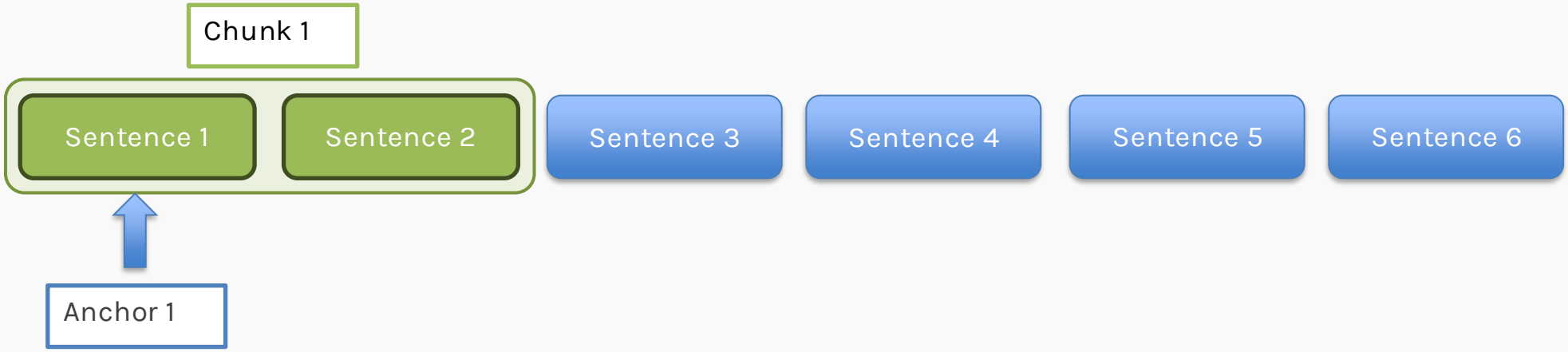
1. Let's suppose we choose the number of sentences at either side to be **1**, this is the window size.
2. We calculate the cosine similarity of the sentences in the group.

Pre-Retrieval Optimization - Indexing



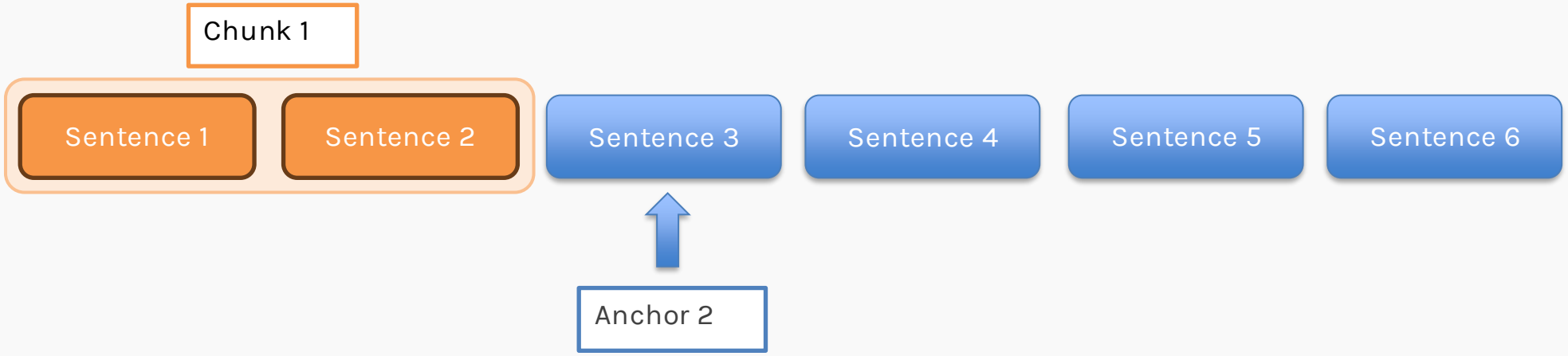
1. Let's suppose we choose the number of sentences at either side to be **1**, this is the window size.
2. We calculate the cosine similarity of the sentences in the group.
3. Since the cosine similarity here is high, we chunk together the sentences.

Pre-Retrieval Optimization - Indexing



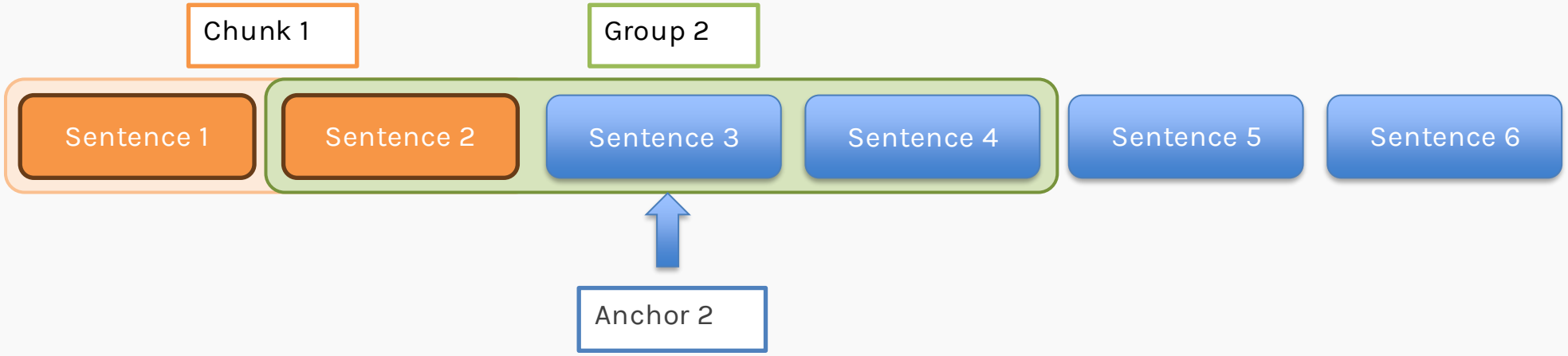
1. Let's suppose we choose the number of sentences at either side to be **1**, this is the window size.
2. We calculate the cosine similarity of the sentences in the group.
3. Since the cosine similarity here is high, we chunk together the sentences.

Pre-Retrieval Optimization - Indexing



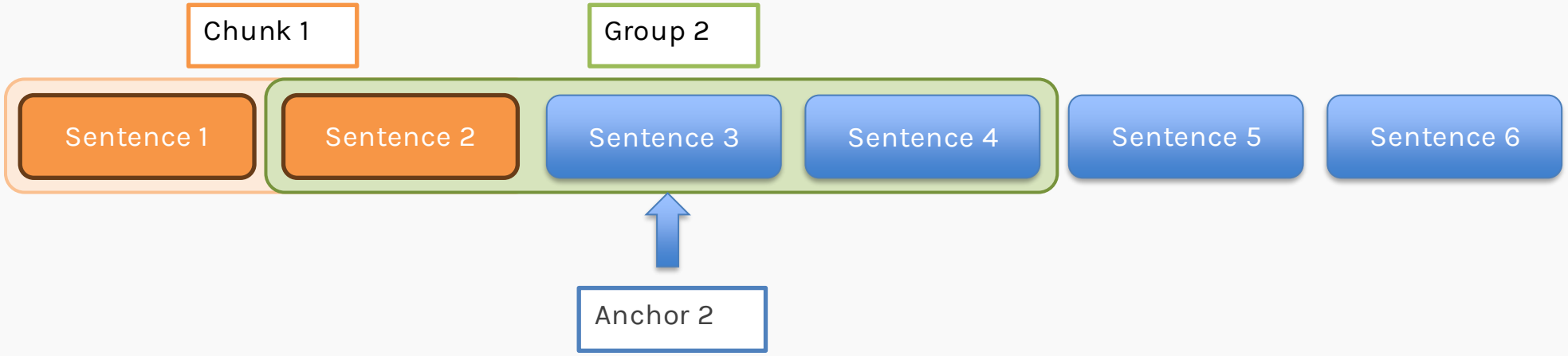
4. We then move to Anchor 2.

Pre-Retrieval Optimization - Indexing



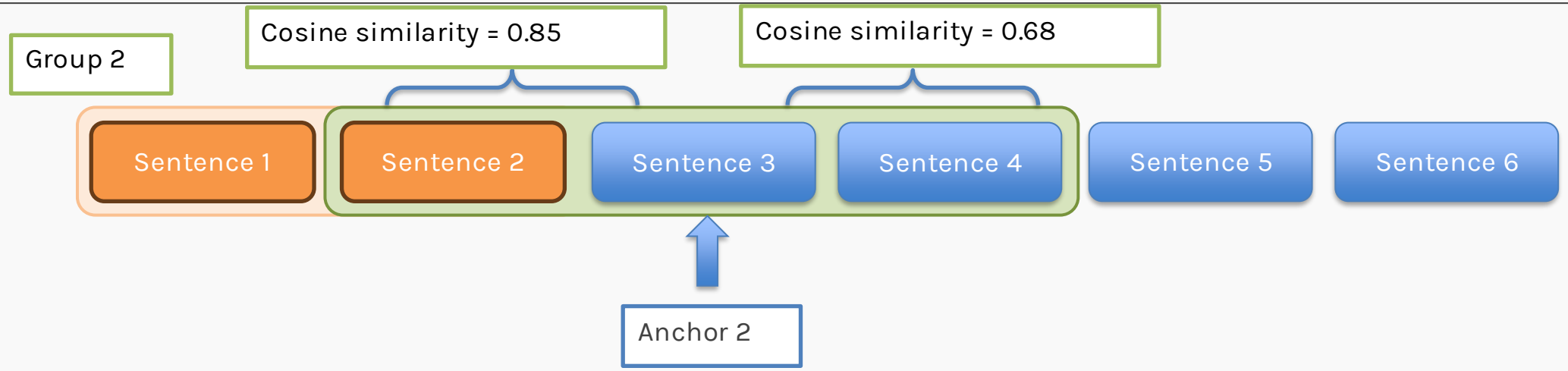
4. We then move to Anchor 2.
5. Since window size=1, we group one sentence from the left and one from the right of the anchor.

Pre-Retrieval Optimization - Indexing



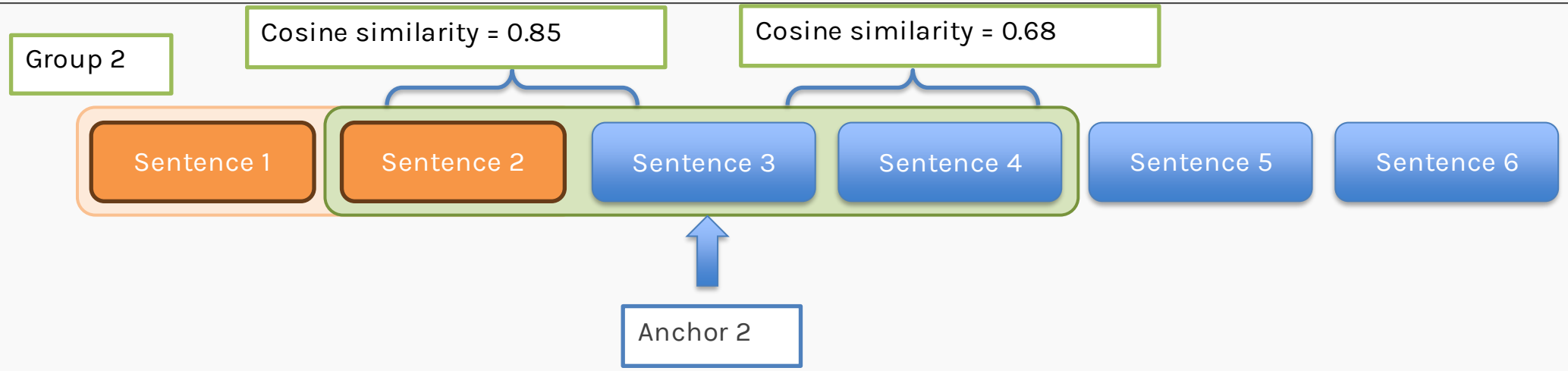
4. We then move to Anchor 2.
5. Since window size=1, we group one sentence from the left and one from the right of the anchor.
6. We then calculate the similarity of the sentences in Group 2 with Anchor 2.

Pre-Retrieval Optimization - Indexing



4. We then move to Anchor 2.
5. Since window size=1, we group one sentence from the left and one from the right of the anchor.
6. We then calculate the similarity of the sentences in Group 2 with Anchor 2.

Pre-Retrieval Optimization - Indexing

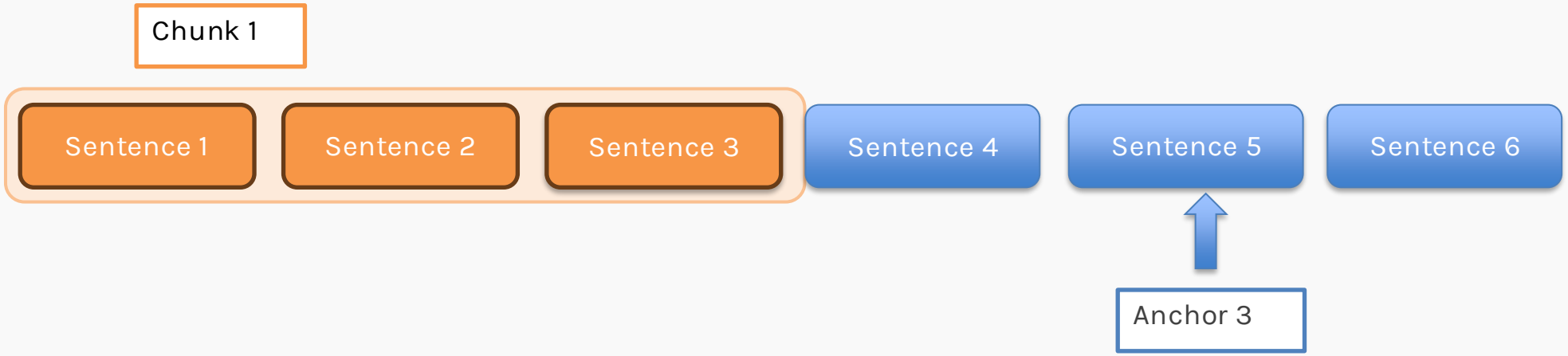


To be considered a high similarity, we defined the threshold to be 0.8. Since,

the similarity of Sentence 2 & Anchor 2 is high and
the similarity of Sentence 4 & Anchor 2 is low

we chunk Sentence 3 into Chunk 1.

Pre-Retrieval Optimization - Indexing



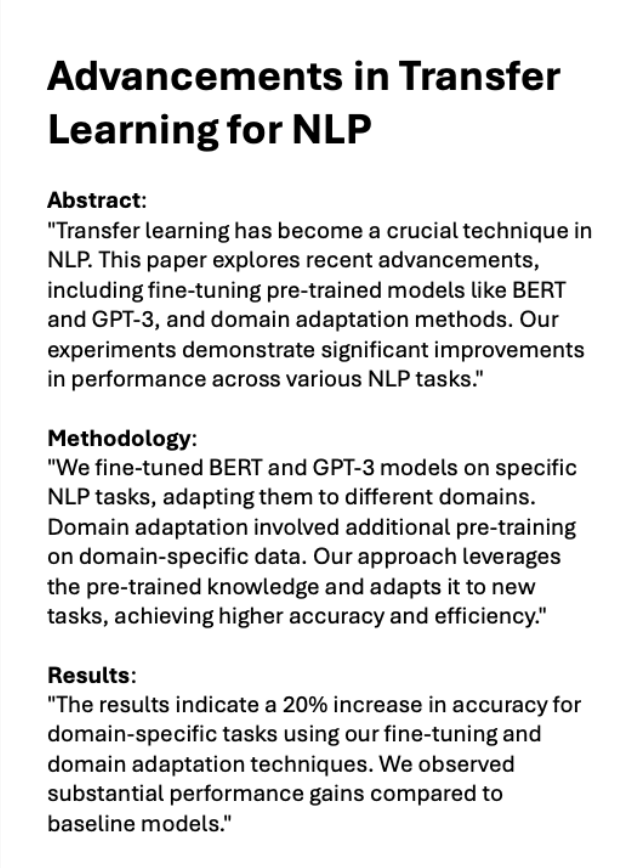
Since Chunk 1 is complete.

We now move on to make the 2nd Chunk.

The anchor moves on to Sentence 5 and the process continues till we reach the end of the token.

Pre-Retrieval Optimization - Indexing

Let's take another example



Advancements in Transfer Learning for NLP

Abstract:
"Transfer learning has become a crucial technique in NLP. This paper explores recent advancements, including fine-tuning pre-trained models like BERT and GPT-3, and domain adaptation methods. Our experiments demonstrate significant improvements in performance across various NLP tasks."

Methodology:
"We fine-tuned BERT and GPT-3 models on specific NLP tasks, adapting them to different domains. Domain adaptation involved additional pre-training on domain-specific data. Our approach leverages the pre-trained knowledge and adapts it to new tasks, achieving higher accuracy and efficiency."

Results:
"The results indicate a 20% increase in accuracy for domain-specific tasks using our fine-tuning and domain adaptation techniques. We observed substantial performance gains compared to baseline models."

Model research paper

The optimal way to do it would be:

Chunk 1:
Advancements in Transfer Learning for NLP

Chunk 2:
Abstract:
Transfer learning has become a crucial technique in NLP. This paper explores recent advancements, including fine-tuning pre-trained models like BERT and GPT-3, and domain adaptation methods. Our experiments demonstrate significant improvements in performance across various NLP tasks.

Chunk 3:
Methodology:
We fine-tuned BERT and GPT-3 models on specific NLP tasks, adapting them to different domains. Domain adaptation involved additional pre-training on domain-specific data. Our approach leverages the pre-trained knowledge and adapts it to new tasks, achieving higher accuracy and efficiency.

....

Pre-Retrieval Optimization – Query Manipulation

Two problems can come up when it comes to queries provided by a user:

1. The query is ‘cluttered’.
   This can be due to it being sprinkled with a lot of irrelevant information.

2. The query is ambiguous.
   The query doesn’t have sufficient information.

Pre-Retrieval Optimization – Query Manipulation

1. The query is ‘cluttered’ for our RAG system

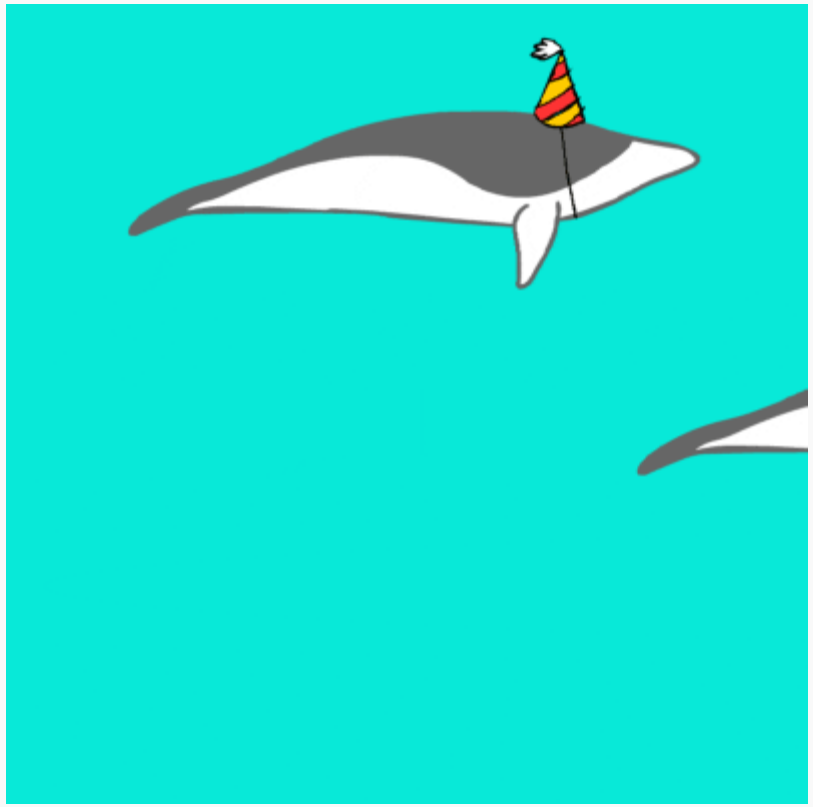
❓Original Query

We have an essay due tomorrow. We have to write about some animal. I love penguins. I could write bout them. But I could also write about dolphins. Are they animals? Maybe. Let’s do dolphins. Where do they live, for example?

🧠Rewritten query

Where do dolphins live?

The rewritten query is now **concise** and **“to the point”**.



Pre-Retrieval Optimization – Query Manipulation

2. The query is ambiguous for our RAG system.

Imagine a use case - we have created a RAG system on top of a Microsoft annual report.

❓Original Query

"Was there significant turnover in the executive team?"

In what sense? Retirements? Promotions?

Too broad a term. Directors? Senior Vice Presidents?

The query lacks information.



Pre-Retrieval Optimization – Query Manipulation

❓Original Query

"Was there significant turnover in the executive team?"

🧠Rewritten query

Was there significant turnover in the executive team? Has there been a notable level of turnover among the executive leadership team recently? Specifically, I am interested in understanding whether multiple key positions within the executive team have experienced changes in leadership, including CEOs, CFOs, or other top executives, over the past year. Additionally, what factors contributed to these changes?

Outline

- Introduction - RAG
- Naïve RAG
- Pre-Retrieval Optimization
- Retrieval Optimization – Hybrid Search
- Post-Retrieval Optimization – Re-ranking
- Self-RAG
- Corrective-RAG
- Fine-tuning (PEFT, LoRA, QLoRA)

Retrieval Optimization – Hybrid Search

Instead of just using semantic search using vectors, we can also do some keyword matching.

For example, if we have the sentence: Chester went to the bank of the river in the morning

Hybrid search in retrieval optimization combines different retrieval models to leverage the strengths of each and provide more relevant search results.

The vector search may help us disambiguate the meaning of the word 'bank' while the keyword matching may help us find documents related to 'Chester'.

Retrieval Optimization – Hybrid Search

How do we do it?

Usually, we use two different retrievers, one for keyword search (BM25) and one for semantic matching (vector similarity).

BM25
A probabilistic retrieval model that ranks documents based on the frequency of query terms in the document.

The formula is based on TF-IDF.

To learn more about BM25: Click here

Retrieval Optimization – Hybrid Search

How do we do it?

Usually, we use ... matching (vec...

BM25
A probabilistic ... in the document.

The formula is based on TF-IDF.

For a word, a higher TF-IDF means a more informative word!
For example:
The word "the" has a high TF, and low IDF. The TF-IDF score implies low importance.
The word "GPT-3" has a low TF, and an extremely high IDF. The TF-IDF score implies high importance.

TF-IDF: A score showing how important a word is in a document.
TF: Term frequency. How often the word appears.
IDF: Inverse document frequency. How rare the word is in the collection.

To learn more about BM25: Click here

Retrieval Optimization – Hybrid Search

How do we combine the results?

We use one of the rank fusion techniques,
such as the **R**eciprocal **R**ank **F**usion (RRF):

$$RRF = \sum_{j=1}^{n} w_j \frac{1}{k + r_j(d)}$$

where,

- n = number of rankings
- $r_j(d)$ = rank of the document
- w_j = weight of the ranking metric
- k = ranking constant

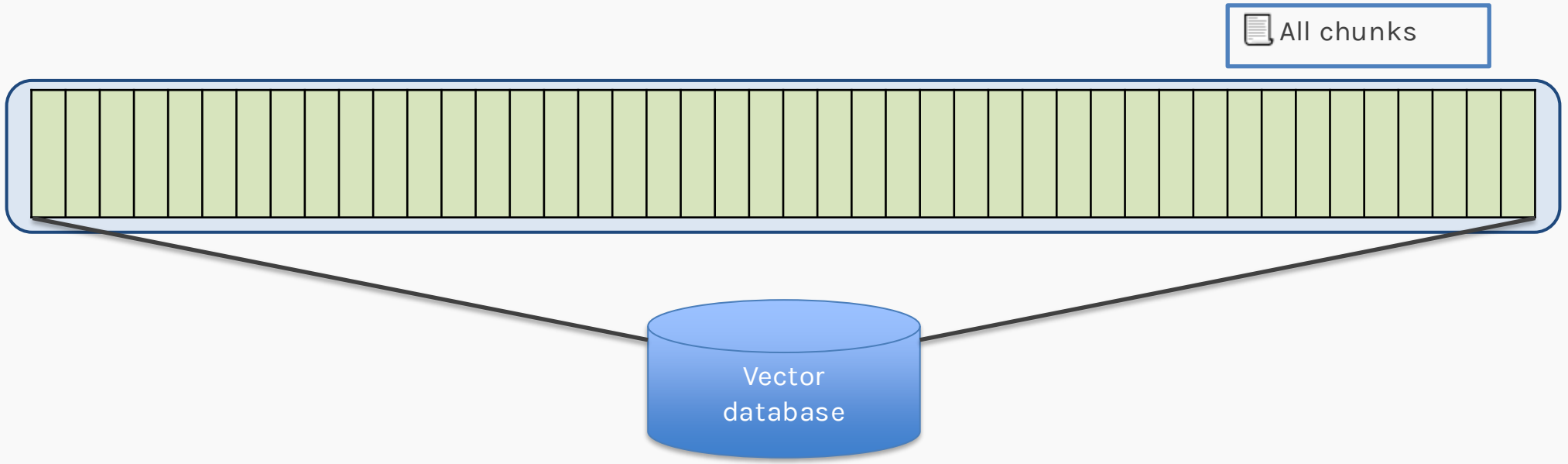
These are hyper-parameters

Outline

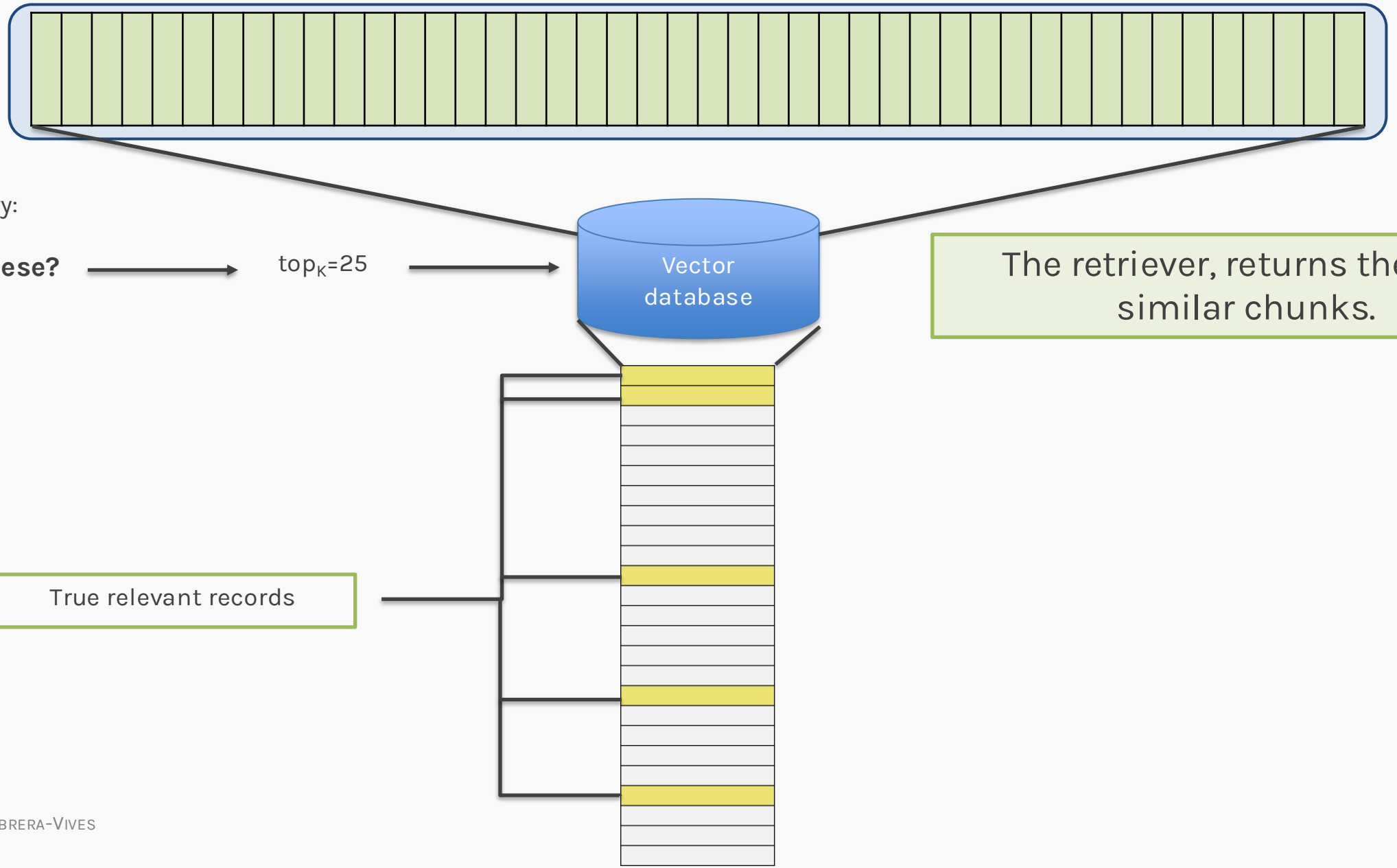
- Introduction - RAG
- Naïve RAG
- Pre-Retrieval Optimization
- Retrieval Optimization – Hybrid Search
- Post-Retrieval Optimization – Re-ranking
- Self-RAG
- Corrective-RAG
- Fine-tuning (PEFT, LoRA, QLoRA)

Post-Retrieval – Re-ranking

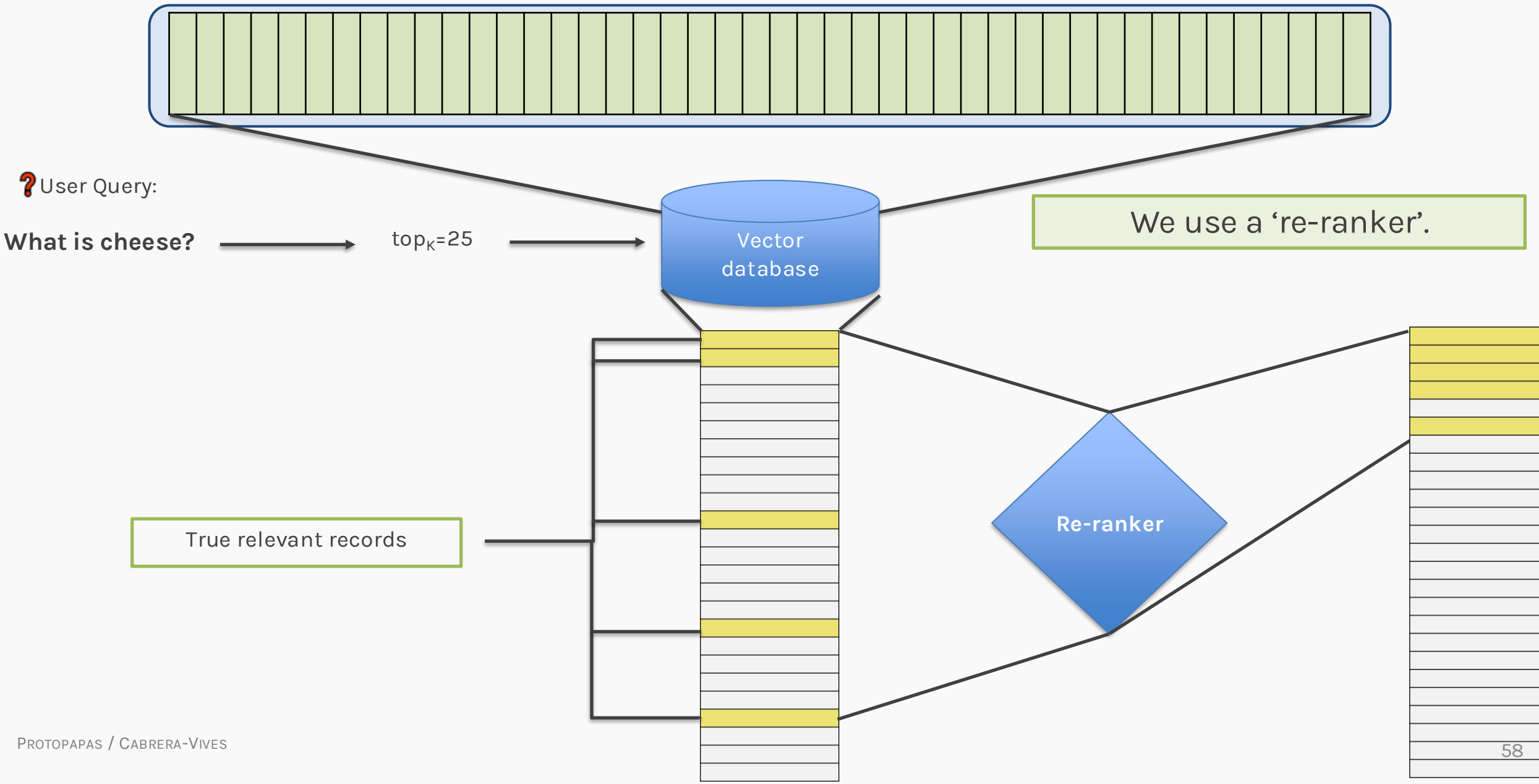
Let us go back to the bigger picture. Consider we have all the chunks stored in our vector database.



Post-Retrieval – Re-ranking



Post-Retrieval – Re-ranking



Post-Retrieval – Re-rankers

1 Why re-rankers

- Retrievers (bi-encoders) compress each document into one vector before the query arrives, so detail is lost.
- The top-k results can miss the most relevant chunk, and the LLM answers from weaker context.

2 What are re-rankers

- A second-stage model that scores each retrieved document against the query.
- Candidates are re-sorted by that score; only the top-n go to the LLM.

3 How they work

A cross-encoder is a model like BERT trained to score a sentence-pair input; it returns a similarity score.



Query and document are read together, in one pass.

4 Why not use them everywhere

- Slow: nothing is precomputed; each query–document pair needs a full model pass.
- 1M documents means 1M BERT passes per query.
- So: retrieve top-k fast, then re-rank only those.

Thus, retrieving and re-ranking mitigates each other's drawbacks (information loss and time).



Outline

- Introduction - RAG
- Naïve RAG
- Pre-Retrieval Optimization
- Retrieval Optimization – Hybrid Search
- Post-Retrieval Optimization – Re-ranking
- Self-RAG
- Corrective-RAG
- Fine-tuning (PEFT, LoRA, QLoRA)

Self-RAG Asai et al., 2023

Limitations of Advanced RAG:

1. Doesn't guarantee the relevancy of the chunk to the query.

2. No guarantee that the response created out of the chunks are related to the chunk itself (hallucinations).

3. Doesn't consider the possibilities where retrieval may not be necessary.

Self-RAG - Intuition

Let's take a scenario where you're taking an open book exam. How would you go about it?

A) For familiar topics, answer quickly; for unfamiliar ones, refer to the book, quickly find the relevant parts, sort and summarize them in our mind, then answer on the exam paper.

B) For every topic, refer to the book. Locate the pertinent sections, mentally collate and summarize them, then write our response on the exam paper.

Self-RAG - Intuition

Referring to the book even when it is not needed can lead to:

1. Slower response rate.

2. More confusion and mistakes.

3. Introduction of irrelevant or erroneous information, whilst scouring through the book.

Similarly, there may be times when it's not required for the RAG to retrieve documents from the vector database.

So, how do we fix this problem?

Self-RAG - Introduction

Self-RAG is a framework that trains and controls the retrieval process via reflection tokens.

Reflection tokens are tokens that are used to improve on the generation and retrieval process.

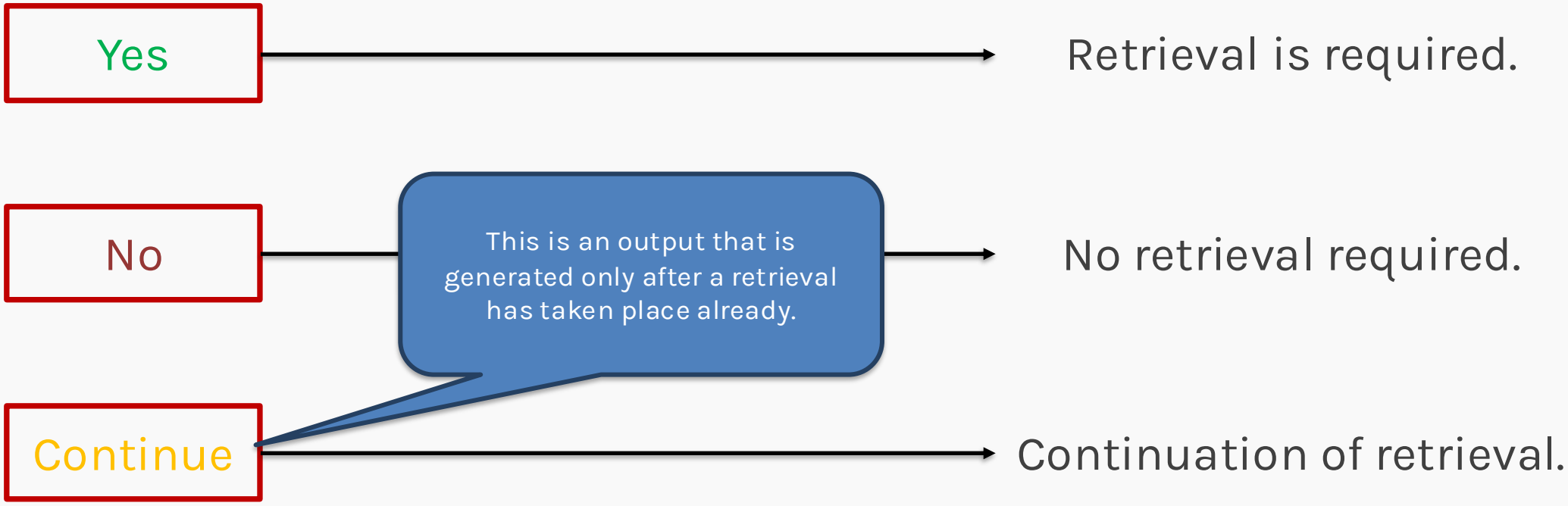
There are two types of reflection tokens:

- Retrieve token: To evaluate the utility of retrieval.
- Critique token: To evaluate the documents that have been retrieved.

Self-RAG – Retrieve token

The retrieve token is generated by the Self-RAG to evaluate the utility of retrieval.

It has 3 possible outputs.



Self-RAG – Critique token

The critique token is generated by the Self-RAG to evaluate the documents that have been retrieved.

It can be further subdivided into 3 types of tokens:

IsREL : Deter…eved document provides useful information to solve the q…

A hallucination check

IsSUP : Determines if the output generated is consistent with or supported by the retrieved document, ensuring that the document justifies the answer.

IsUSE : Determines if the output generated effectively answers or addresses the query (if the output generated was useful to the query).

Self-RAG

Type	Input	Possible Values	Definition
Retrieve	(Query) or (Query, retrieved chunk, and previous segments – if any)	{yes, no, continue}	Decided if to use the retriever
IsREL	Query, Retrieved Chunk	{relevant, irrelevant}	If chunk proves useful information to solve query
IsSUP	Query, Retrieved Chunk, Current Output	{fully supported, partially supported, no support}	If current segment answer is supported by the chunk
IsUSE	Query, Current Output	{5, 4, 3, 2, 1}	If current output is a useful response to the query

Outline

- Introduction - RAG
- Naïve RAG
- Pre-Retrieval Optimization
- Retrieval Optimization – Hybrid Search
- Post-Retrieval Optimization – Re-ranking
- Self-RAG
- Corrective-RAG
- Fine-tuning (PEFT, LoRA, QLoRA)

Corrective-RAG



Yan et al., 2024

Corrective-RAG

When reading a book, we often come across information which is insufficient or ambiguous.

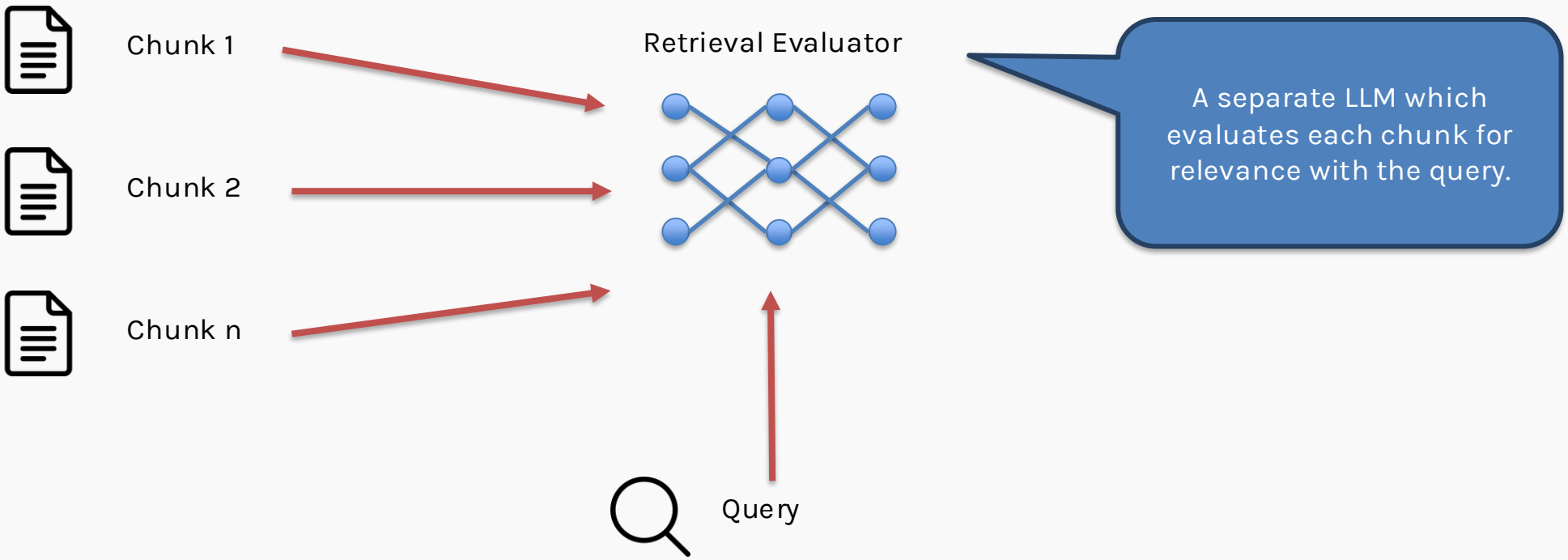
So, what do we do then?

Solution: We refer to the internet for additional details.

That's exactly what our next variant of RAG does!

Corrective-RAG

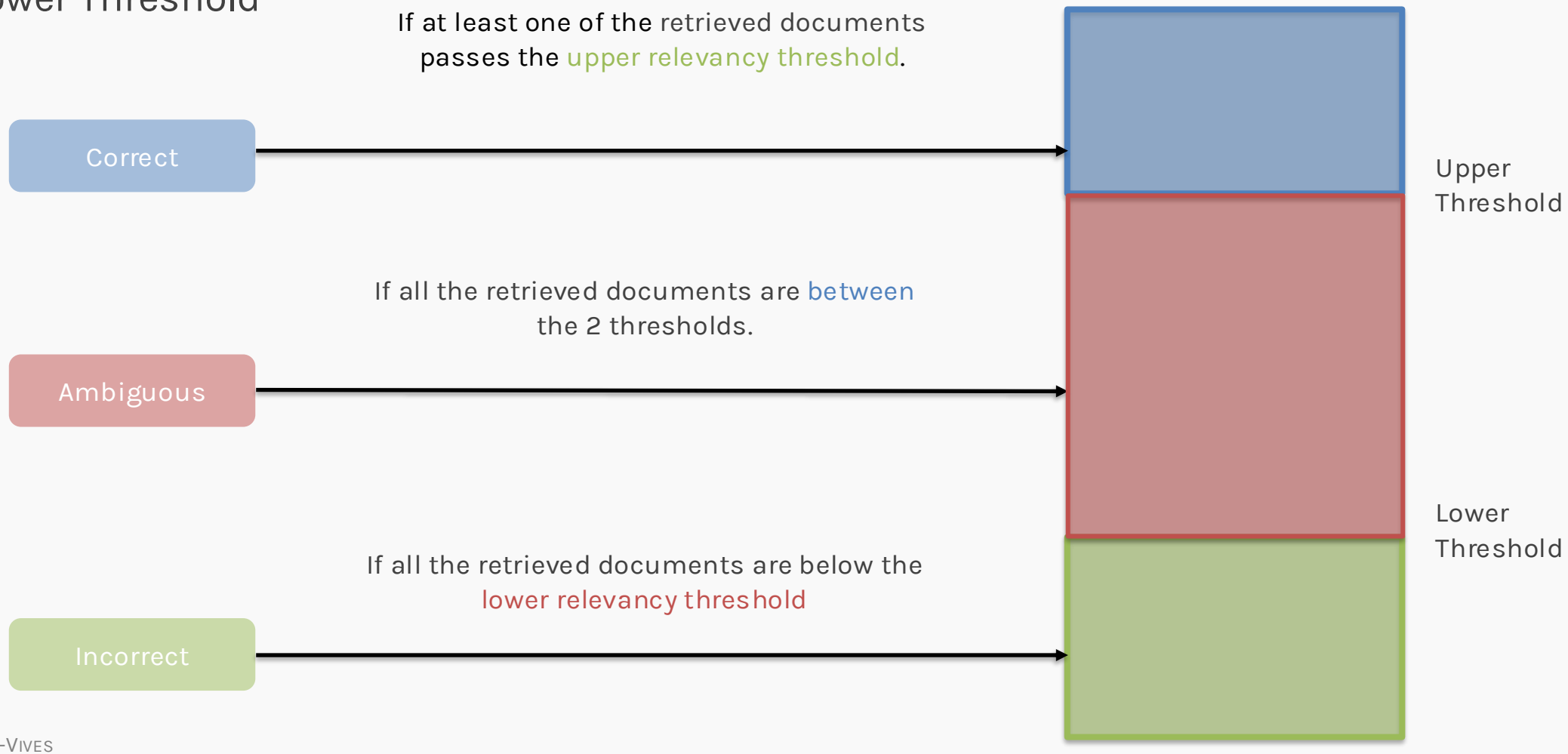
Let's suppose these are the chunks/documents we got after we retrieve and re-rank:



Corrective-RAG

The 3 outputs are given based on 2 thresholds which are set beforehand.

1. Upper Threshold
2. Lower Threshold



Corrective-RAG

Correct

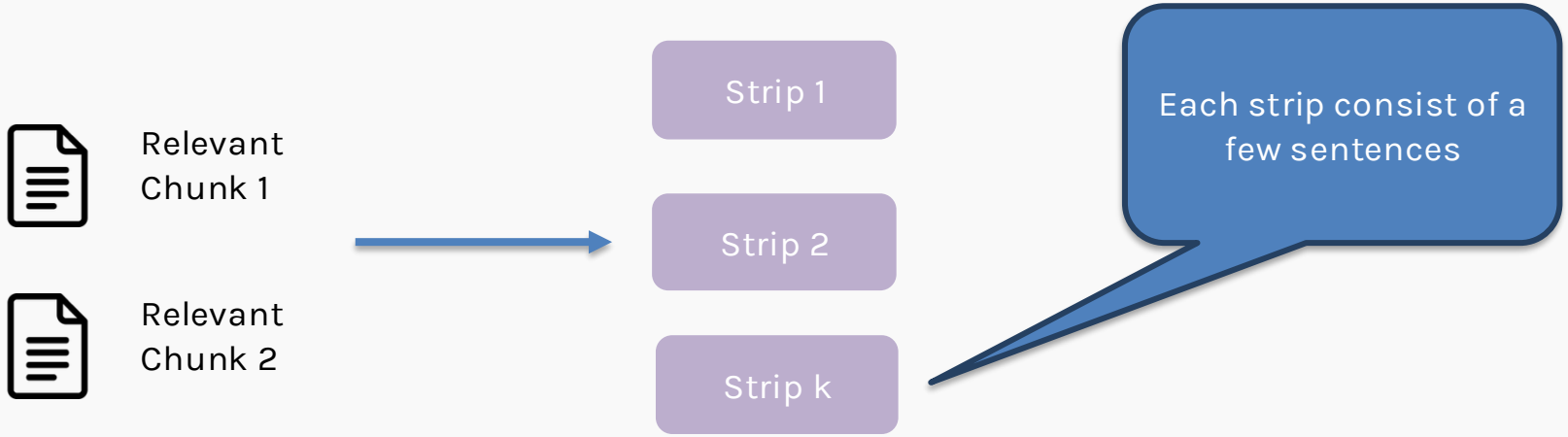
Ambiguous

Incorrect

Correct:

This is when the chunks are relevant to the query provided.

In this case, we do knowledge refinement.



Corrective-RAG

Correct

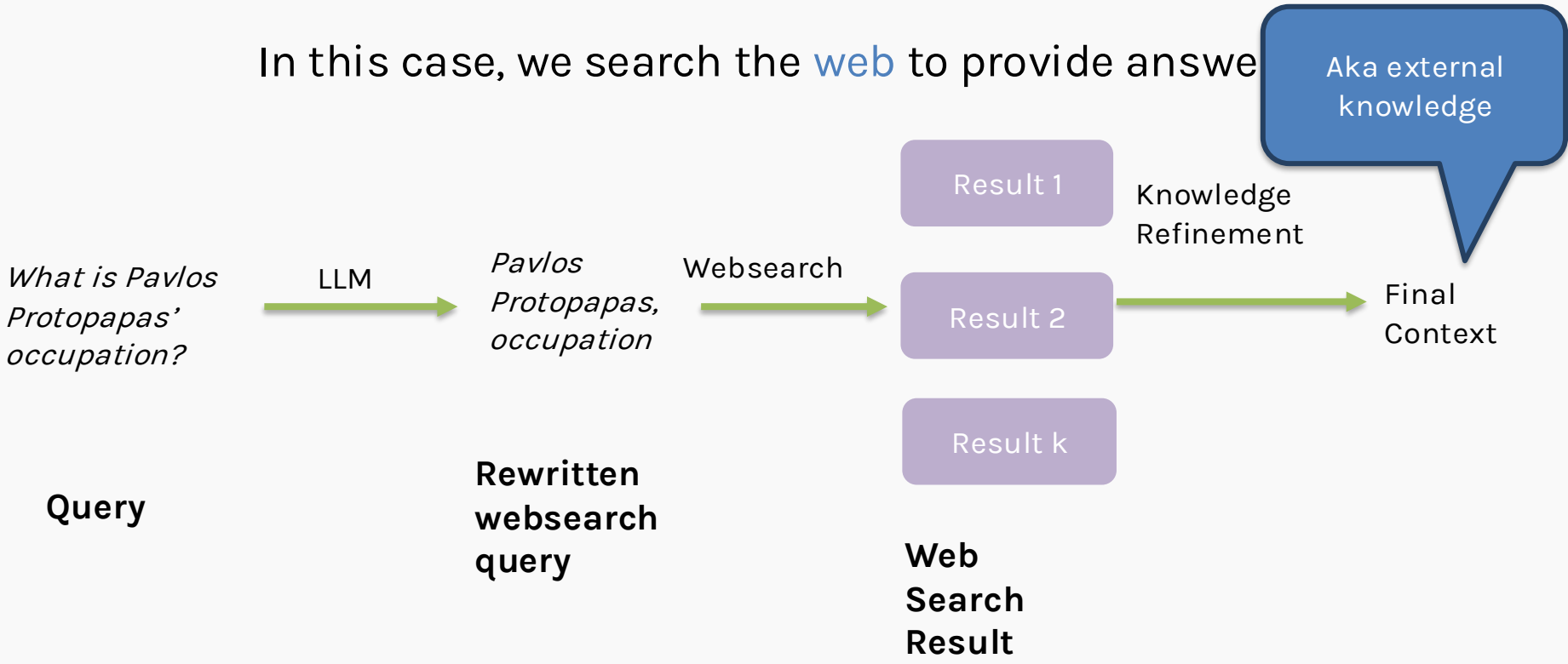
Ambiguous

Incorrect

Incorrect:

This is when no retrieved chunk is relevant to the query.

In this case, we search the web to provide answe



Corrective-RAG

Correct

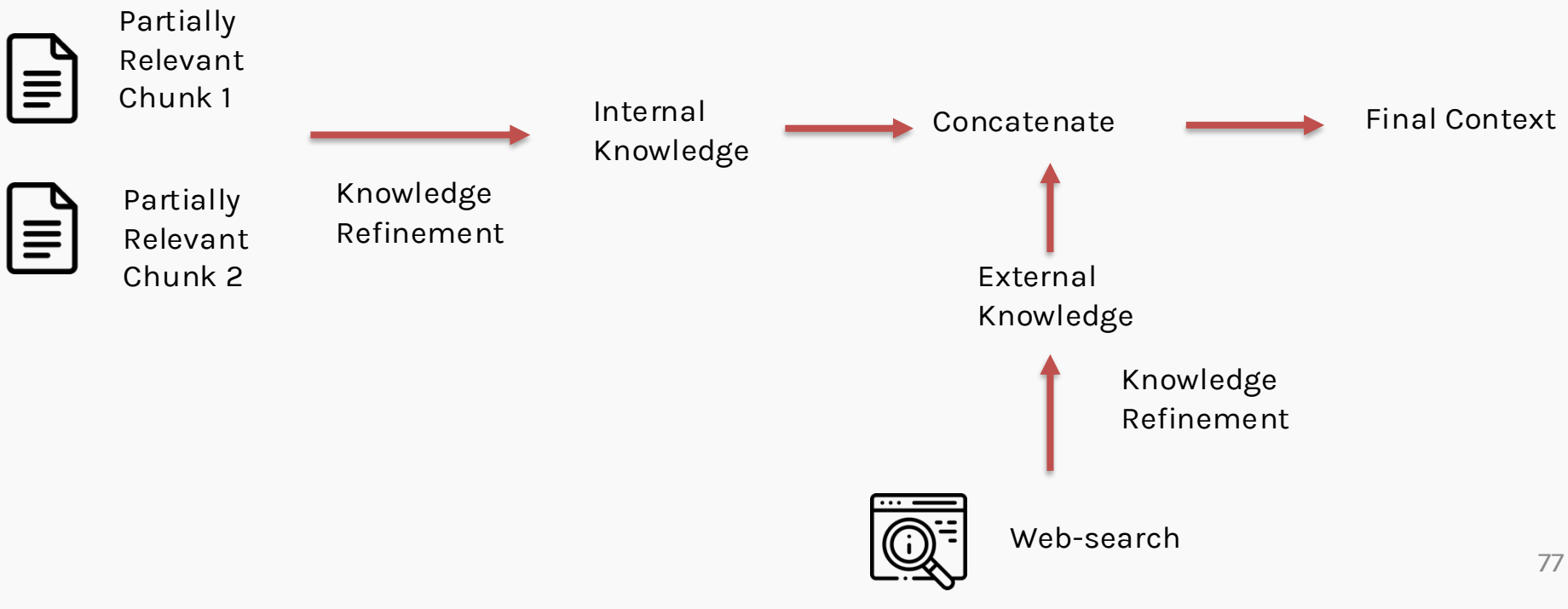
Ambiguous

Incorrect

Ambiguous:

This is when the retrieved chunks aren't correct or incorrect

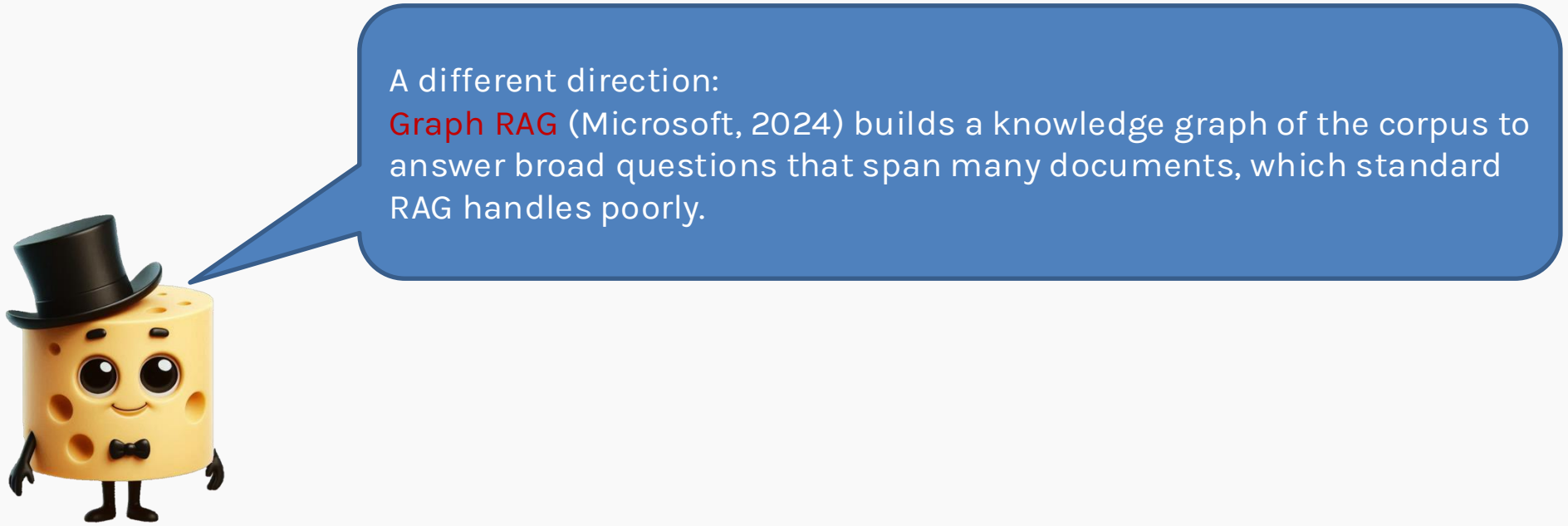
In this case, we combine both the internal and external knowledge to create our final context



Corrective-RAG

- Corrective RAG is plug and play and can be combined with naive RAG, advanced RAG, and even Self-RAG.

- When we combine corrective RAG with self-RAG, we get Self-CRAG, which beat both on the paper's benchmarks (2024).



Outline

- Introduction - RAG
- Naïve RAG
- Pre-Retrieval Optimization
- Retrieval Optimization – Hybrid Search
- Post-Retrieval Optimization – Re-ranking
- Self-RAG
- Corrective-RAG
- Fine-tuning (PEFT, LoRA, QLoRA)

The other lever: change the model, not the context

Prompting

- Instructions + examples in the context
- Instant, no data, no training
- Limited by what the model already knows and by context length

RAG

- Retrieve facts at question time
- Fresh and private knowledge, with citations
- Quality depends on chunking, retrieval and re-ranking

Fine-tuning

- Change the weights: style, format, domain language, behaviour
- Needs a dataset, a GPU and an evaluation set
- Does not reliably add facts — pair it with RAG for knowledge

Rule of thumb: **facts → RAG · behaviour → fine-tune · both → RAG on top of a small fine-tuned model.**

Instruction-tuning (Full Parameter)

Fine-tuning very often means instruction fine-tuning.

An instruction dataset, comprising pairs of instructions, answers, and sometimes context, is required for such fine-tuning.

Instruction-tuning (Full Parameter)

- We need to update all the parameters while finetuning.
  - For a 7B model, we need to update 7 billion weights. For a 13 billion model, we need to update 13 billion weights and so on.

- Storing and updating these weights require a lot of GPU memory.

 **Fun Fact:** Did you know, training GPT-4 involved **~25,000 A100 GPUs** over **~90-100 days**, costing OpenAI nearly **$100 million**!

Instruction-tuning (Full Parameter)

This makes full parameter finetuning inaccessible to normal folks like us.

So, what can we do?



Instruction-tuning (PEFT)

PEFT stands for Parameter Efficient Finetuning.

Unlike full parameter finetuning, PEFT preserves the vast majority of the model's original weights.

There are majorly three methods to do PEFT.

1. Additive
2. Selective
3. Reparameterization

LoRA - Introduction

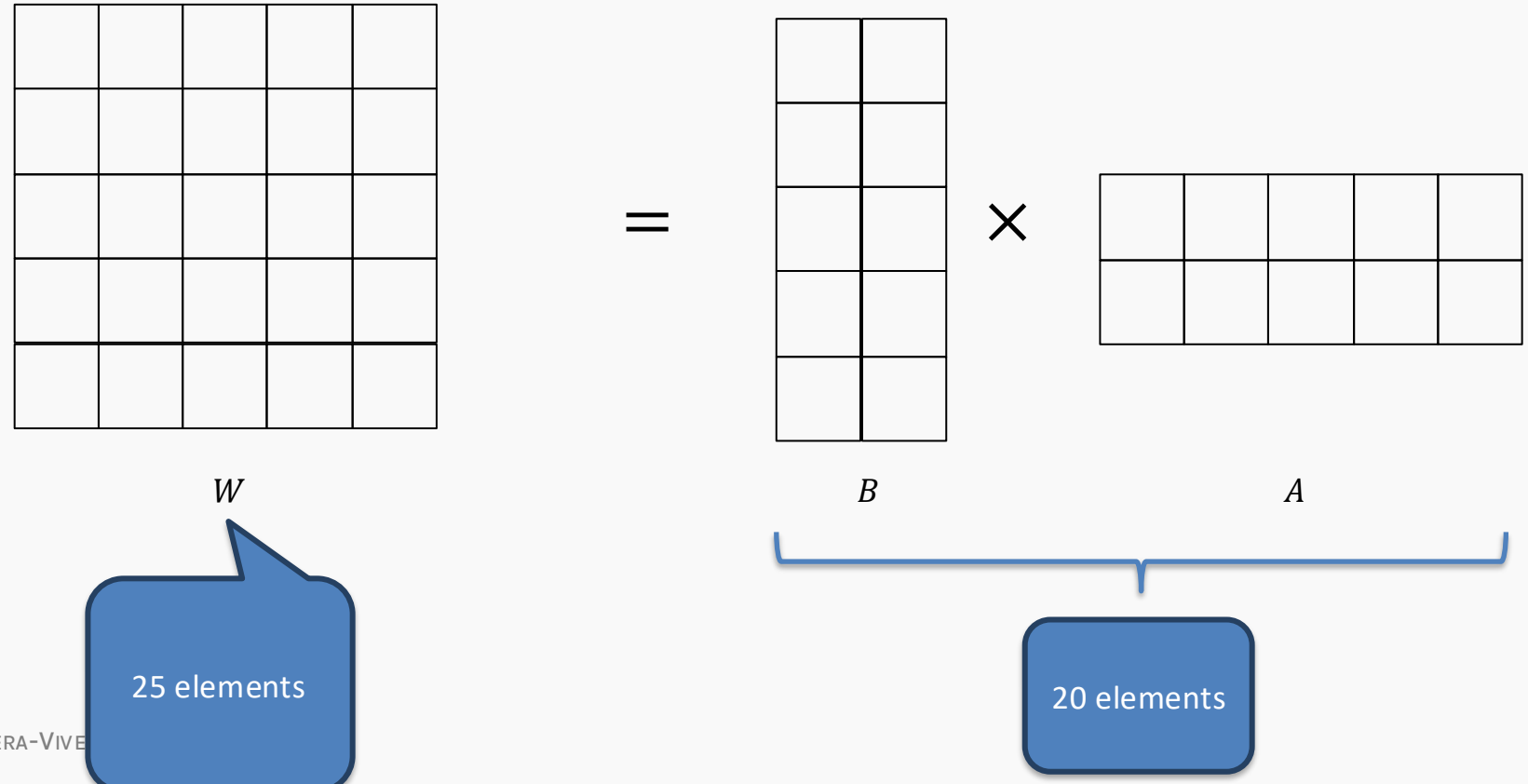
LoRA is a training method for LLMs that reduces the number of trainable parameters and memory requirements, while still achieving good performance.

LoRA works by adding pairs of rank decomposition weight matrices (called update matrices) to each layer of the LLM.

These update matrices have significantly lower parameters than the original model weights, and only the update matrices are trained.

LoRA - Intuition

We can even increase the rank to get better performance.



LoRA - Summary

- LoRA reduces the trainable parameters and memory requirements while maintaining good performance.
- LoRA adds pairs of rank decomposition weight matrices (called update matrices) to each layer of the LLM.
- Only the update matrices, which have significantly fewer parameters than the original model weights, are trained.

Summary

Change the context: RAG

Naïve RAG	Chunk and embed documents into a vector database, retrieve the top-k chunks and add them to the prompt.
Pre-retrieval	Improve indexing and rewrite the query before searching.
Retrieval	Hybrid search combines keyword and embedding matches.
Post-retrieval	Re-rankers (cross-encoders) re-score the top candidates for precision.
Self-RAG	The model decides when to retrieve and critiques its own output with reflection tokens.
Corrective RAG	An evaluator grades retrieved documents; weak results trigger web search and refinement.

Change the model: fine-tuning

Full-parameter tuning
Updates every weight on instruction–answer pairs; the most GPU-hungry option.

PEFT
Freezes the base model and trains a small set of added or selected parameters.

LoRA
Learns low-rank update matrices that merge back into the weights, so inference costs nothing extra.

DAY 1 · THURSDAY 22 OCTOBER

Retrieval-Augmented Generation

Naïve RAG, better retrieval, Self-RAG and Corrective RAG · optional: fine-tuning



Pavlos Protopapas
Harvard University

Guillermo Cabrera-Vives
Universidad de Concepción